# [UNIVERSITY NAME]

**School of Computer Science Engineering & Technology**

*[ University Logo ]*

*A Mini-Project Report on*

# LEGACY CIPHER MIGRATION FRAMEWORK FOR INDUSTRIAL CONTROL SYSTEMS

*A Protocol-Translation Gateway from Classical Ciphers to AES-256-GCM for Smart-Grid Telemetry, with a Study of Transition Vulnerabilities*

Submitted in partial fulfilment of the requirements of the course

**[Course Code] - [Course Name]**

[Programme, e.g. B.Tech. (CSE)], [Semester]

**Submitted by**


| Name | Enrolment No. |
|---|---|
| [Student Name 1] | [Enrolment No. 1] |
| [Student Name 2] | [Enrolment No. 2] |
| [Student Name 3] | [Enrolment No. 3] |


**Under the guidance of**

[Faculty Name]

[Designation, Department]

**SCHOOL OF COMPUTER SCIENCE ENGINEERING & TECHNOLOGY**

**[UNIVERSITY NAME], [CITY]**

**November 2026**

# CERTIFICATE

This is to certify that the mini-project report entitled **"Legacy Cipher Migration Framework for Industrial Control Systems"** submitted by [Student Name 1] ([Enrolment No. 1]), [Student Name 2] ([Enrolment No. 2]) and [Student Name 3] ([Enrolment No. 3]) in partial fulfilment of the requirements of the course [Course Code] - [Course Name], [Programme, e.g. B.Tech. (CSE)], [Semester], School of Computer Science Engineering & Technology, [University Name], is a bonafide record of the work carried out by them under my supervision during the academic year 2026-27.

To the best of my knowledge, the work presented in this report is original and has not been submitted elsewhere for the award of any degree, diploma or course credit.

| | |
|---|---|
| Signature of the Supervisor | Signature of the Head / Coordinator |
| [Faculty Name] | Name: |
| [Designation, Department] | Date: |

# DECLARATION

We hereby declare that the mini-project report entitled **"Legacy Cipher Migration Framework for Industrial Control Systems"** is an authentic record of our own work, carried out during the academic year 2026-27 under the guidance of [Faculty Name], School of Computer Science Engineering & Technology, [University Name].

The matter embodied in this report has not been submitted by us for the award of any other degree, diploma or course credit. All sources of information, standards and published work used in this report have been duly acknowledged and cited. The source code, experiments and results presented here were developed by the team members named below.

| **Name** | **Enrolment No.** | **Signature** |
|---|---|---|
| [Student Name 1] | [Enrolment No. 1] | _____ |
| [Student Name 2] | [Enrolment No. 2] | _____ |
| [Student Name 3] | [Enrolment No. 3] | _____ |

Place: [City]

Date:

# ACKNOWLEDGEMENT

We express our sincere gratitude to our project supervisor, [Faculty Name], [Designation, Department], for the guidance, encouragement and constructive feedback provided throughout this mini-project. The discussions on cryptographic design and on the realities of industrial control systems shaped both the direction and the rigour of this work.

We are thankful to the School of Computer Science Engineering & Technology, [University Name], for providing the academic environment, computing facilities and the opportunity to undertake this project as part of the course [Course Code] - [Course Name].

We also thank the authors and maintainers of the open-source tools on which this project relies - Python, the pyca/cryptography library, pytest and matplotlib - and the standards bodies whose publicly available specifications made it possible to verify our implementations.

Finally, we thank our classmates for their feedback during review sessions, and our families for their constant support.

**[Student Name 1], [Student Name 2], [Student Name 3]**

# ABSTRACT

Electric-grid substations still contain Remote Terminal Units (RTUs) whose telemetry links are unencrypted or protected only by classical substitution and transposition ciphers. Because field devices remain in service for decades, utilities modernise gradually, and legacy and modern cryptography must coexist for years. This project presents LCMF, a Legacy Cipher Migration Framework for Industrial Control Systems, which (i) reproduces a smart-grid environment of eight 11 kV feeders protected by Caesar, Additive, Affine, Vigenere, Playfair and Columnar transposition ciphers; (ii) provides a bidirectional protocol-translation gateway that wraps legacy telemetry into a 68-byte AES-256-GCM Secure Telemetry Envelope with an authenticated header, per-device HKDF keys, crash-safe nonces and anti-replay protection; and (iii) systematically studies the security of the transition itself.

AES-128/192/256 and the ECB, CBC, CTR and GCM modes were implemented from first principles and validated against FIPS-197 and GCM test vectors, and against the cryptography library on 500 random cases. Exploiting the public frame grammar, every legacy cipher was broken from at most two captured frames in under a second, including a Playfair cipher with a nominal key space of 2^138. Eight transition vulnerabilities were demonstrated and quantified: false data laundered through the gateway from the legacy last mile, downgrade attacks in hybrid operation, replay laundering, malleability of confidentiality-only AES, ECB pattern leakage, GCM nonce reuse after reboots, AES keys derived from legacy secrets, and bandwidth overhead. For each, a mitigation implemented in the framework - device pinning, legacy sequence tracking, authenticated encryption, persistent counters, HKDF key hierarchy and plausibility filtering - was measured; all reduced attack success to zero except physically consistent false data on the legacy last mile, which only replacing the RTU removes. The gateway translated 14,400 frames without error at a median latency of 34.9 microseconds and 53 bytes of overhead per reading. The results are condensed into an evidence-based migration playbook for utilities.

**Keywords:** industrial control systems, SCADA, smart grid, legacy cipher migration, classical cryptanalysis, AES-GCM, protocol-translation gateway, transition vulnerabilities.

# TABLE OF CONTENTS

Certificate ..... i

Declaration ..... ii

Acknowledgement ..... iii

Abstract ..... iv

List of Figures ..... ix

List of Tables ..... xi

List of Abbreviations ..... xiii

Chapter 1: Introduction ..... 1

1.1 Background ..... 1

1.2 Motivation ..... 1

1.3 Problem Statement ..... 2

1.4 Objectives ..... 2

1.5 Scope and Limitations ..... 3

1.6 Contributions ..... 3

1.7 Organisation of the Report ..... 4

Chapter 2: Literature Review ..... 5

2.1 Foundations of Classical Cryptography ..... 5

2.2 Monoalphabetic Ciphers ..... 5

2.2.1 Additive (shift) and Caesar ciphers ..... 5

2.2.2 Affine cipher ..... 5

2.2.3 Cryptanalysis of monoalphabetic ciphers ..... 6

2.3 Polyalphabetic Ciphers ..... 6

2.3.1 Vigenere cipher ..... 6

2.3.2 Playfair cipher ..... 6

2.4 Transposition Ciphers ..... 6

2.5 The Advanced Encryption Standard ..... 7

2.6 Modes of Operation and Authenticated Encryption ..... 7
2.7 ICS Protocols and their Security Extensions ..... 8
2.8 Existing Approaches to Securing Legacy SCADA Links ..... 8
2.9 Research Gap ..... 9
Chapter 3: Problem Analysis, Requirements and Feasibility ..... 10
3.1 Problem Domain Analysis ..... 10
3.2 Threat Model ..... 10
3.3 Stakeholders and Use Cases ..... 11
3.4 Functional Requirements ..... 11
3.5 Non-Functional Requirements ..... 12
3.6 Feasibility Study ..... 12
3.6.1 Technical feasibility ..... 12
3.6.2 Economic feasibility ..... 13
3.6.3 Operational feasibility ..... 13
3.6.4 Schedule feasibility ..... 13
3.6.5 Legal and ethical feasibility ..... 13
3.7 Hardware and Software Requirements ..... 14
Chapter 4: System Design and Methodology ..... 15
4.1 Methodology ..... 15
4.2 System Architecture ..... 15
4.3 Data Design ..... 17
4.3.1 Legacy telemetry frame ..... 17
4.3.2 Modern payload ..... 17
4.3.3 Secure Telemetry Envelope (STE) ..... 18
4.4 Design of the Legacy Cipher Subsystem ..... 18
4.5 Design of the Modern Cryptography Engine ..... 19
4.6 Gateway Pipeline and Migration Policy ..... 20

4.7 Security Analysis and Experiment Design ..... 22
4.8 Module and Class Design ..... 23
4.9 Justification of Algorithms and Techniques ..... 23
Chapter 5: Implementation ..... 25
5.1 Development Environment and Project Structure ..... 25
5.2 Legacy Cipher Suite and Cryptanalysis (Member 1) ..... 25
5.3 Modern Cryptography Engine and Secure Envelope (Member 2) ..... 26
5.4 Grid Simulator, Gateway and SCADA (Member 3) ..... 27
5.5 Design-to-Implementation Traceability ..... 29
5.6 Testing ..... 30
5.7 Demonstration of the Modules ..... 30
Chapter 6: Results and Discussion ..... 32
6.1 Experimental Setup ..... 32
6.2 Correctness of the Cryptographic Implementations ..... 32
6.3 Cryptanalysis of the Legacy Ciphers ..... 33
6.4 Correctness and Performance of the Gateway ..... 35
6.5 Transition Vulnerabilities ..... 37
6.5.1 TV1 - Legacy last mile and trust laundering ..... 37
6.5.2 TV2 - Downgrade attack during hybrid operation ..... 37
6.5.3 TV3 - Replay and freshness laundering ..... 38
6.5.4 TV4 - Malleability: AES alone is not enough ..... 39
6.5.5 TV5 - Pattern leakage with ECB ..... 39
6.5.6 TV6 - Nonce reuse after a reboot ..... 40
6.5.7 TV7 - AES keys derived from legacy secrets ..... 40
6.5.8 TV8 - Bandwidth and latency overhead ..... 41
6.6 Risk and Mitigation Matrix ..... 42
6.7 Achievement of Objectives ..... 43

6.8 Discussion ..... 43
Chapter 7: Conclusion and Future Scope ..... 45
7.1 Conclusion ..... 45
7.2 Limitations ..... 45
7.3 Future Scope ..... 45
References ..... 47
Appendix A: User Manual ..... 50
Appendix B: Individual Contributions ..... 51
Appendix C: Sample Output of the Demonstration ..... 53

# LIST OF FIGURES

**Figure 3.1:** Project schedule (12 weeks) and allocation of work streams to team members ..... 13

**Figure 4.1:** System architecture of LCMF with adversary positions A1 and A2 ..... 16

**Figure 4.2:** Data-flow diagram, level 0 (context diagram) ..... 16

**Figure 4.3:** Legacy Telemetry Frame (LTF): fixed tags (known plaintext) and variable fields ..... 17

**Figure 4.4:** Byte layout of the Secure Telemetry Envelope (STE v1) ..... 18

**Figure 4.5:** AES-256 encryption: round structure and key expansion as implemented in aes_core.py ..... 19

**Figure 4.6:** AES-GCM encryption and tag computation ..... 20

**Figure 4.7:** Uplink processing pipeline from RTU to SCADA historian, with rejection points ..... 21

**Figure 4.8:** Migration phases and the state of each device ..... 21

**Figure 4.9:** Package and class design of LCMF (owner of each package in brackets) ..... 23

**Figure 6.1:** Nominal key space of each cipher compared with the measured effort to break it ..... 34

**Figure 6.2:** Symbol frequency profiles and index of coincidence of plaintext and ciphertext ..... 34

**Figure 6.3:** Playfair digram-codebook attack: decryption coverage versus known frames ..... 35

**Figure 6.4:** Gateway uplink latency per legacy cipher with the library AES-GCM engine ..... 36

**Figure 6.5:** The same bitmap encrypted with AES-ECB (structure visible) and with GCM's counter mode (noise) ..... 40

**Figure 6.6:** Serial-link time to poll eight RTUs, legacy frames versus secure envelopes ..... 41

**Figure 6.7:** Attack success before and after mitigation for each transition vulnerability ..... 42

# LIST OF TABLES

**Table 1.1:** Measurable objectives of the project ..... 2

**Table 2.1:** Comparison of existing approaches with the proposed framework..... 9

**Table 3.1:** Analysis of the problem domain ..... 10

**Table 3.2:** Principal use cases ..... 11

**Table 3.3:** Functional requirements ..... 11

**Table 3.4:** Non-functional requirements and their metrics ..... 12

**Table 3.5:** Hardware and software requirements ..... 14

**Table 4.1:** Fields of the legacy telemetry frame..... 17

**Table 4.2:** Legacy ciphers configured in the simulated network ..... 18

**Table 4.3:** Migration policies enforced at the control centre ..... 21

**Table 4.4:** Cryptanalysis design per legacy cipher ..... 22

**Table 4.5:** Design of the transition-vulnerability experiments ..... 22

**Table 4.6:** Justification of the main algorithms and techniques..... 23

**Table 5.1:** Size of the implementation (non-blank lines)..... 25

**Table 5.2:** Traceability matrix: requirement - design - implementation - verification - evidence ..... 29

**Table 5.3:** Automated test suites ..... 30

**Table 5.4:** Command-line demonstration of each module ..... 30

**Table 6.1:** Experimental setup ..... 32

**Table 6.2:** Validation against published test vectors..... 32

**Table 6.3:** Key recovery against every legacy cipher from captured traffic..... 33

**Table 6.4:** End-to-end delivery and data integrity in each migration mode ..... 35

**Table 6.5:** Micro-benchmarks (median time per operation) ..... 36

**Table 6.6:** TV1 - false-data injection on the legacy last mile ..... 37

**Table 6.7:** TV2 - downgrade attack against each migration policy ..... 38

**Table 6.8:** TV3 - replay attacks..... 38

**Table 6.9:** TV4 - blind tampering of the breaker status without the key ..... 39

**Table 6.10:** TV5 - inference of breaker state from ciphertext alone ..... 39

**Table 6.11:** TV6 - GCM nonce management across reboots ..... 40

**Table 6.12:** TV7 - key provenance ..... 41

**Table 6.13:** TV8 - link time per frame and per 8-RTU scan ..... 41

**Table 6.14:** Risk assessment of the transition vulnerabilities (L = low, M = medium, H = high) ..... 42

**Table 6.15:** Objectives, targets and measured results ..... 43

**Table A.1:** Commands provided by the framework ..... 50

**Table B.1:** Contribution of each team member ..... 51

**Table B.2:** Task responsibility matrix ..... 51

**Table B.3:** Code, report and presentation ownership ..... 52

# LIST OF ABBREVIATIONS

| | |
|---|---|
| AAD | Additional Authenticated Data |
| AEAD | Authenticated Encryption with Associated Data |
| AES | Advanced Encryption Standard |
| CBC | Cipher Block Chaining |
| CTR | Counter mode |
| DFD | Data-Flow Diagram |
| DNP3 | Distributed Network Protocol 3 (IEEE 1815) |
| ECB | Electronic Codebook |
| FDIA | False-Data Injection Attack |
| FIPS | Federal Information Processing Standard |
| GCM | Galois/Counter Mode |
| GF | Galois Field |
| HKDF | HMAC-based Key Derivation Function |
| HMAC | Hash-based Message Authentication Code |
| ICS | Industrial Control System |
| IEC | International Electrotechnical Commission |
| IoC | Index of Coincidence |
| IV | Initialisation Vector |
| LCMF | Legacy Cipher Migration Framework (this project) |
| LTF | Legacy Telemetry Frame |
| MAC | Message Authentication Code |
| NIST | National Institute of Standards and Technology |
| OT | Operational Technology |
| RTU | Remote Terminal Unit |
| SCADA | Supervisory Control and Data Acquisition |
| STE | Secure Telemetry Envelope |
| TV | Transition Vulnerability |
| WAN | Wide-Area Network |
| Z36 | 36-symbol alphabet A-Z, 0-9 |

# CHAPTER 1: INTRODUCTION

## 1.1 Background

Electric power grids are supervised and controlled by Industrial Control Systems (ICS). In a typical utility, a Supervisory Control and Data Acquisition (SCADA) master station in the control centre periodically polls Remote Terminal Units (RTUs) installed in substations. Each RTU reports telemetry - bus voltage, feeder current, system frequency, active power and the status of circuit breakers - and executes control commands such as opening or closing a breaker [1], [2]. The integrity and timeliness of this exchange is what allows operators to balance generation with demand, isolate faults and restore supply.

Much of this communication infrastructure was designed decades ago, when the control network was assumed to be physically isolated. Links were built from serial lines, leased telephone circuits and licensed radio running at 1,200 to 19,200 baud. Where any protection existed it was usually a lightweight substitution or transposition scheme - effectively a classical cipher - chosen because it was cheap to implement in early microcontrollers, and much of the security rested on the obscurity of proprietary formats [3], [4]. The smart-grid programme has since connected these islands to IP networks, enterprise systems and the Internet, removing the isolation that the original designs relied on [5], [6].

Field equipment in a grid has a service life of fifteen to thirty years, and replacing every RTU in a utility is expensive and operationally disruptive [1]. Utilities therefore modernise gradually, often by inserting *bump-in-the-wire* devices or protocol gateways in front of legacy equipment [7], [8]. For years, sometimes decades, legacy and modern cryptography coexist in the same network. This project studies exactly that coexistence.

## 1.2 Motivation

Attacks on power systems are no longer hypothetical. In December 2015, attackers who had gained access to the SCADA environments of three Ukrainian distribution companies remotely opened breakers and left approximately 225,000 customers without power [9]. Stuxnet demonstrated that manipulated process data and commands can cause physical damage while operators see normal values [10], and false-data-

injection attacks can mislead the state estimators that grid operators depend on [11], [12].

Classical ciphers offer no meaningful defence against such adversaries: their key spaces are tiny or their structure leaks the statistics of the plaintext [13], [14], [15]. Replacing them with the Advanced Encryption Standard (AES) is clearly necessary. However, for a utility the practical question is not whether AES is stronger than a Caesar cipher - it obviously is - but **what can go wrong while old and new systems coexist, and how a migration can be carried out safely**. That question motivates this project.

## 1.3 Problem Statement

*Design and implement a protocol-translation gateway that wraps legacy, unencrypted or classically encrypted smart-grid telemetry into AES-encrypted payloads, and systematically identify, demonstrate and quantify the security vulnerabilities that arise during the transition from legacy to modern cryptography, together with measured mitigations.*

The problem has three coupled parts: (i) faithfully reproducing the legacy environment - its ciphers, frame formats and grid behaviour - so that conclusions are meaningful; (ii) building a gateway that is correct, fast enough for real-time operation and cryptographically sound; and (iii) evaluating the *transition* itself, not just the end state, by attacking every intermediate configuration that a real migration passes through.

## 1.4 Objectives

Each objective is stated with a measurable acceptance target. Table 1.1 lists them; Chapter 6 reports the measured value for each one.

**Table 1.1:** Measurable objectives of the project

| ID | Objective | Acceptance target |
|---|---|---|
| O1 | Implement six legacy ciphers correctly | 100 % of textbook vectors; 0 round-trip failures on 10,000 frames per cipher |
| O2 | Implement AES-128/192/256 and GCM from first principles | 100 % of FIPS-197 / GCM vectors; bit-identical to library on 500 cases |
| O3 | Translate legacy telemetry to AES-GCM without data loss or delay | 0 corrupted readings in all 4 modes; median gateway latency < 1 ms |
| O4 | Break every legacy cipher from captured traffic | key recovered from <= 3 frames in < 1 s for each cipher |

| ID | Objective | Acceptance target |
|---|---|---|
| O5 | Secure configuration blocks forged, replayed, tampered and downgraded frames | 0 % acceptance in every STRICT / PINNED / GCM scenario |
| O6 | Identify and quantify transition vulnerabilities with mitigations | >= 8 vulnerabilities, each with a measured mitigation |
| O7 | Keep the secure envelope within legacy-link budgets | overhead <= 64 B/frame; 8-device scan at 9600 baud < 2 s cycle |

## 1.5 Scope and Limitations

**In scope.** A software framework, named LCMF (Legacy Cipher Migration Framework), comprising: six legacy ciphers (Caesar, Additive, Affine, Vigenere, Playfair and keyed Columnar transposition) plus unencrypted links; AES-128/192/256 and the ECB, CBC, CTR and GCM modes implemented from first principles; a production AES-256-GCM path using an audited library; a simulated 11 kV distribution network with eight feeders, legacy RTUs, an upgraded RTU and a SCADA master; a bidirectional protocol-translation gateway; four migration policies; and a reproducible experiment suite covering cryptanalysis of the legacy ciphers and eight transition vulnerabilities.

**Out of scope.** Physical hardware; byte-exact implementations of Modbus, DNP3 or IEC 60870-5-104 (their telemetry is abstracted into a representative fixed-width legacy frame); compromise of the gateway or SCADA host; side-channel attacks; and public-key infrastructure. All attacks are executed only against the project's own simulator.

## 1.6 Contributions

- A modular, tested implementation of the classical cipher suite over a 36-symbol telemetry alphabet, including a 6x6 Playfair variant that enciphers digits losslessly.
- A from-scratch AES and GCM implementation that is bit-identical to the cryptography library on all published test vectors and on 500 random cases.
- A protocol-translation gateway and a 68-byte Secure Telemetry Envelope (STE) whose 37-byte header is authenticated as associated data.
- Practical key-recovery attacks that break every legacy cipher from at most two captured frames by exploiting the public frame grammar.

- A catalogue of eight transition vulnerabilities (TV1-TV8), each demonstrated in the simulator, quantified, and paired with a mitigation whose effect is measured.
- A fully reproducible pipeline: one command regenerates all results, figures, this report and the presentation.

## 1.7 Organisation of the Report

Chapter 2 reviews classical and modern cryptography, ICS protocols and existing approaches to securing legacy SCADA links. Chapter 3 analyses the problem domain, defines the threat model and requirements, and establishes feasibility. Chapter 4 presents the design and methodology, and Chapter 5 the implementation together with a design-to-implementation traceability matrix. Chapter 6 reports and discusses the results, and Chapter 7 concludes with limitations and future scope. Appendix A is a user manual, Appendix B documents the contribution of each team member, and Appendix C reproduces sample output of the demonstration.

# CHAPTER 2: LITERATURE REVIEW

This chapter reviews the cryptographic primitives that the project migrates from and to, the security extensions available for ICS protocols, and existing systems for retrofitting legacy SCADA links. It closes by identifying the research gap addressed in this work.

## 2.1 Foundations of Classical Cryptography

A cipher is a pair of functions E and D such that D_K(E_K(P)) = P for every key K. Kerckhoffs's principle states that a system must remain secure even if everything about it except the key is public [16]. Classical ciphers fall into two families: *substitution* ciphers replace symbols, and *transposition* ciphers permute their positions. Substitution ciphers are *monoalphabetic* when one fixed mapping is used for the whole message and *polyalphabetic* when the mapping changes with position [13], [14]. Throughout this report a symbol is identified with its index in an alphabet of m symbols (m = 26 for A-Z, m = 36 for the telemetry alphabet A-Z0-9), and arithmetic is modulo m.

## 2.2 Monoalphabetic Ciphers

### 2.2.1 Additive (shift) and Caesar ciphers

The additive cipher adds a secret key k to each symbol:

$$C = (P + k) \bmod m, \quad P = (C - k) \bmod m \tag{2.1}$$

The Caesar cipher is the historical special case k = 3 [15]. Because the key is fixed into the algorithm, Caesar has no secret once the method is known (key space 1); the general additive cipher has only m keys and is broken by trying all of them.

### 2.2.2 Affine cipher

The affine cipher combines multiplication and addition:

$$C = (a \cdot P + b) \bmod m, \quad P = a^{-1}(C - b) \bmod m, \quad \gcd(a, m) = 1 \tag{2.2}$$

The multiplier must be invertible modulo m, so a ranges over the φ(m) units of Z_m. For m = 26 and m = 36 alike, φ(m) = 12, giving 12 × 26 = 312 and 12 × 36 = 432 keys respectively [14]. Two known plaintext-ciphertext pairs with an invertible difference determine the key by solving a pair of linear congruences.

### 2.2.3 Cryptanalysis of monoalphabetic ciphers

Since each plaintext symbol always maps to the same ciphertext symbol, the frequency profile of the plaintext is merely relabelled. Frequency analysis compares observed counts O_i with expected counts E_i using the chi-squared statistic, and the shift or key that minimises it is selected [17]:

$$\chi^2 = \Sigma\, (O_i - E_i)^2 / E_i \tag{2.3}$$

## 2.3 Polyalphabetic Ciphers

### 2.3.1 Vigenere cipher

The Vigenere cipher applies a different shift to each position, cycling through a keyword of length L:

$$C_i = (P_i + K_(i \bmod L)) \bmod m \tag{2.4}$$

Its key space m^L grows quickly with L, but the cipher is periodic. Kasiski observed that repeated plaintext fragments encrypted under the same key position produce repeated ciphertext fragments whose distances are multiples of L [18]. Friedman's index of coincidence (IoC) measures how far a text is from uniform; for English it is about 0.066, for uniformly random A-Z text 1/26 = 0.038 [19]. Splitting the ciphertext into L columns with the correct period restores English-like IoC in every column, after which each column is a shift cipher solved by chi-squared.

$$IoC = \Sigma\, n_i(n_i - 1) / [N(N - 1)] \tag{2.5}$$

### 2.3.2 Playfair cipher

Playfair, a digraphic cipher, enciphers pairs of letters using a 5x5 square built from a keyword (I and J share a cell). For a pair in the same row each letter is replaced by its right neighbour; in the same column by the letter below; otherwise by the letters at the other corners of their rectangle. Doubled letters within a pair are separated by a filler, normally X [13]. The nominal key space is 25! (about 2^84), but Playfair leaks digram statistics, maps reversed digrams to reversed ciphertext digrams, and yields readily to known-plaintext attacks [15].

## 2.4 Transposition Ciphers

A keyed columnar transposition writes the plaintext row-wise into L columns and reads the columns out in the order defined by the alphabetical rank of the key letters. The key space is L!, and because only positions change, the symbol frequencies of the ciphertext

are identical to those of the plaintext - a property that immediately reveals that a transposition has been used. Historically, such ciphers were attacked by anagramming and by exhaustive search over column orders [17].

## 2.5 The Advanced Encryption Standard

AES is the Rijndael block cipher standardised by NIST in FIPS 197 [20], [21]. It encrypts 128-bit blocks under 128-, 192- or 256-bit keys using 10, 12 or 14 rounds. The 16-byte state is arranged as a 4x4 matrix of bytes, and each round applies four transformations:

- **SubBytes** - a non-linear byte substitution: the multiplicative inverse in GF(2^8) modulo x^8 + x^4 + x^3 + x + 1, followed by an affine transformation over GF(2).
- **ShiftRows** - row r of the state is rotated left by r bytes, mixing bytes across columns.
- **MixColumns** - each column is multiplied by a fixed polynomial matrix over GF(2^8), giving diffusion within a column.
- **AddRoundKey** - the state is XORed with a round key derived by the key-expansion routine (RotWord, SubWord and round constants).

The final round omits MixColumns. After more than two decades of public analysis no practical attack on full AES is known, which is why it is the default choice for protecting critical-infrastructure data [1], [22].

## 2.6 Modes of Operation and Authenticated Encryption

A block cipher on its own encrypts only one 16-byte block; a *mode of operation* defines how longer messages are processed [23]:

- **ECB** encrypts every block independently, so identical plaintext blocks give identical ciphertext blocks and patterns remain visible.
- **CBC** chains each block with the previous ciphertext block under a random IV. It provides confidentiality only; flipping IV bits flips the corresponding first-block plaintext bits, and padding errors can create decryption oracles [24].
- **CTR** turns the cipher into a stream cipher. It is fast and parallel but perfectly malleable: flipping a ciphertext bit flips the same plaintext bit.

- **GCM** combines CTR encryption with GHASH, a polynomial hash over GF(2^128), to produce a 128-bit authentication tag over the ciphertext and any associated data (AAD) [25], [26].

Modern guidance is to use authenticated encryption with associated data (AEAD) such as GCM rather than confidentiality-only modes [22], [27]. GCM has one critical requirement: a (key, nonce) pair must never repeat. A repeated nonce reveals the XOR of the two plaintexts and allows recovery of the authentication subkey H, which enables forgeries [28], [29]. Supporting mechanisms used in this project are the HMAC-based key derivation function HKDF [30] and the sliding anti-replay window of IPsec ESP [31].

## 2.7 ICS Protocols and their Security Extensions

Most field protocols in use today were designed without security. Modbus has no authentication at all; DNP3 (IEEE 1815) added Secure Authentication, a challenge-response HMAC mechanism that authenticates critical messages but does not encrypt them [32], [33]. IEC 62351 specifies TLS profiles for TCP-based protocols such as IEC 60870-5-104 and DNP3 over IP, and message authentication for IEC 61850 [34]. The Modbus/TCP Security specification wraps Modbus in mutually authenticated TLS [35]. IEC 62443 provides the overarching zone-and-conduit security architecture for industrial automation [36]. All of these, however, require the end devices to support the new protocol - which a legacy serial RTU cannot do.

## 2.8 Existing Approaches to Securing Legacy SCADA Links

Because legacy devices cannot be upgraded in place, several retrofit approaches have been proposed. The American Gas Association's AGA-12 effort defined cryptographic protection for serial SCADA links using inline modules [7] and evolved into IEEE 1711.2, the Secure SCADA Communications Protocol [8]. Wright et al. studied how to provide such protection with low latency on slow links [37], and Tsang and Smith's YASIR proposed a low-latency, high-integrity retrofit specifically for legacy serial SCADA [38]. Fovino et al. designed a secure variant of Modbus [39]. In practice, utilities also tunnel traffic through IPsec VPNs between sites. Table 2.1 compares these approaches.

**Table 2.1:** Comparison of existing approaches with the proposed framework

| Approach | Retrofits legacy RTU? | Integrity / authenticity | Main limitation |
|---|---|---|---|
| Classical cipher on the link (status quo) | n/a | None (checksum only) | Broken from one or two frames (Section 6.3) |
| DNP3 Secure Authentication [32] | No - needs firmware support | HMAC challenge-response | No confidentiality; device upgrade required |
| IEC 62351 / Modbus-TCP Security (TLS) [34], [35] | No - needs TCP/IP stack with TLS | Yes (TLS) | Not applicable to serial legacy devices |
| AGA-12 / IEEE 1711.2 bump-in-the-wire [7], [8] | Yes - inline modules | Yes | Module needed at both ends; transition risks not quantified |
| YASIR [38] | Yes | Yes, low latency | Research prototype; focuses on the link, not the migration |
| IPsec / VPN tunnel | Partly - protects WAN only | Yes | Serial last mile and LAN stay in clear |
| **LCMF (this work)** | **Yes - translation gateway** | **AES-256-GCM, header as AAD, anti-replay, policy pinning** | **Last mile stays legacy until the RTU is replaced - quantified as TV1** |

## 2.9 Research Gap

The literature treats the problem in two separate ways: cryptographers analyse ciphers and modes in isolation, and ICS researchers design secure end-state protocols or individual retrofit devices. Comparatively little work quantifies the risks *during* the migration, when legacy and modern links coexist: the unprotected last mile behind a gateway, downgrade to legacy protocols, replays that are laundered into fresh secure messages, misuse of AES modes by integrators, nonce management on devices that reboot, and keys that are derived from old legacy secrets. This project closes that gap by building a migration framework and measuring each of these hazards experimentally.

# CHAPTER 3: PROBLEM ANALYSIS, REQUIREMENTS AND FEASIBILITY

## 3.1 Problem Domain Analysis

Grid telemetry differs from typical IT traffic in ways that drive every design decision of this project. Table 3.1 relates each property of the domain to its security consequence and to the resulting design implication.

**Table 3.1:** Analysis of the problem domain

| Domain property | Security consequence | Design implication |
|---|---|---|
| Small, fixed-format frames (tens of bytes) | Fixed tags are *known plaintext*: 24 of 52 legacy symbols never change | Assume Kerckhoffs-style attacker; legacy ciphers must be evaluated against known-plaintext attacks |
| Periodic polling every few seconds | Old frames are valid-looking and can be replayed | Sequence numbers, replay window and timestamp freshness |
| Slowly varying values, frequent repeats | Low entropy - deterministic encryption leaks patterns | Never use ECB; one nonce per message |
| Low-bandwidth serial / radio links (1.2-19.2 kbaud) | Cryptographic overhead costs scan time | Compact binary payload; measure overhead per baud rate |
| Physical consequences of commands | Integrity matters more than confidentiality | Authenticated encryption; plausibility checks |
| 15-30 year device lifetime | Legacy and modern links coexist for years | Explicit migration phases and policies; study the transition |
| Availability is the top OT priority [1] | Security must not drop legitimate data | Measure delivery rate and latency in every mode |

The analysis shows that the weakness of a classical cipher on a telemetry link is foremost an *integrity* problem: an attacker who recovers the key can inject commands or falsify measurements. Confidentiality matters as well - telemetry reveals load, outages and switching operations - but the most severe consequences are physical.

## 3.2 Threat Model

Two adversary positions are considered, labelled A1 and A2 in the architecture of Chapter 4:

- **A1 - legacy last mile.** The adversary taps the serial or radio link between an RTU and the gateway (for example at an unmanned substation or a radio repeater).

- **A2 - wide-area network.** The adversary is on the path between the gateway (or an upgraded RTU) and the control centre.

At either position the adversary can eavesdrop, record, replay, inject, modify and drop frames. Following Kerckhoffs's principle, the adversary knows the cipher families, the frame format and the protocol behaviour, but not the keys, and is computationally bounded. Compromise of the gateway or SCADA hosts, physical tampering with RTUs, side channels and malicious insiders holding keys are outside the scope of this study. The assets to protect are, in order of priority, the integrity of commands, the integrity and freshness of telemetry, the availability of legitimate data, and the confidentiality of operational information.

## 3.3 Stakeholders and Use Cases

**Table 3.2:** Principal use cases

| ID | Actor | Use case | Outcome |
|---|---|---|---|
| UC1 | Legacy / upgraded RTU | Report telemetry each poll | Reading stored in the SCADA historian |
| UC2 | SCADA operator | Open or close a breaker | Command executed by the addressed RTU only |
| UC3 | Utility engineer | Move the network to the next migration phase | Policy mode and device states updated |
| UC4 | Security analyst | Assess the strength of a legacy cipher | Key-recovery result with effort metrics |
| UC5 | Security analyst | Run transition-vulnerability experiments | results/*.json, figures and report regenerated |
| UC6 | Key administrator | Rotate keys | New key generation used; old envelopes still verifiable |

## 3.4 Functional Requirements

**Table 3.3:** Functional requirements

| ID | Requirement |
|---|---|
| FR1 | Encrypt and decrypt telemetry with Caesar, Additive, Affine, Vigenere, Playfair and Columnar ciphers, and support unencrypted legacy links. |
| FR2 | Encode and decode legacy telemetry and command frames, including checksum validation. |
| FR3 | Simulate eight 11 kV feeders producing physically consistent telemetry, with breaker control. |

| ID | Requirement |
|---|---|
| FR4 | Translate legacy uplink frames into AES-256-GCM Secure Telemetry Envelopes (gateway uplink). |
| FR5 | Translate authenticated envelope commands into legacy command frames (gateway downlink). |
| FR6 | Enforce one of four migration policies at the control centre. |
| FR7 | Reject replayed envelopes (sequence window and timestamp) and replayed legacy frames at the gateway. |
| FR8 | Check every decoded reading for physical plausibility before it is sealed. |
| FR9 | Recover the keys of the legacy ciphers from captured traffic for security assessment. |
| FR10 | Run the transition-vulnerability experiments TV1-TV8 reproducibly and store the results. |
| FR11 | Provide a command-line demonstration and regenerate results, figures, report and slides. |

## 3.5 Non-Functional Requirements

**Table 3.4:** Non-functional requirements and their metrics

| ID | Quality | Metric / target |
|---|---|---|
| NFR1 | Performance | Median gateway latency per frame < 1 ms |
| NFR2 | Bandwidth efficiency | Envelope overhead <= 64 bytes; 8-RTU scan at 9,600 baud within a 2 s cycle |
| NFR3 | Correctness | 0 corrupted readings end to end in every migration mode |
| NFR4 | Security | AES-256, 96-bit unique nonces, 128-bit tags, per-device per-direction keys |
| NFR5 | Reproducibility | All experiments seeded; one command regenerates every number |
| NFR6 | Maintainability | Modular packages with a common cipher interface; automated tests |
| NFR7 | Portability | Pure Python 3.10+ with one runtime dependency; Windows and Linux |

## 3.6 Feasibility Study

### 3.6.1 Technical feasibility

Every algorithm used is a published standard or textbook construction, and mature open-source implementations exist for the production path [40]. A preliminary benchmark on the development laptop (Intel64 Family 6 Model 140 Stepping 1, GenuineIntel) showed AES-256-GCM sealing a reading in about 1.0 microseconds, more than 6 orders of magnitude below the 2 s polling interval. The project is therefore

technically feasible on commodity hardware; the final measured median gateway latency is 34.9 microseconds (Section 6.4).

### 3.6.2 Economic feasibility

The software stack is entirely open source (Python, the cryptography library, pytest and matplotlib), so there is no licensing cost. More importantly, the gateway approach keeps the existing RTUs in service: the capital cost of migration is limited to one gateway per substation instead of replacing every field device, and the same software runs on any industrial PC. Upgraded RTUs can be introduced later, as budgets allow, without changing the control centre.

### 3.6.3 Operational feasibility

The gateway is transparent to both sides: RTUs keep speaking their legacy protocol and the SCADA master receives the same readings. Migration proceeds in phases that can be rolled back, and the envelope overhead keeps an eight-RTU scan at 9,600 baud at 567 ms, well inside a 2 s scan cycle (Section 6.5.8). Operators therefore need no retraining.

### 3.6.4 Schedule feasibility

The work was planned over twelve weeks with three parallel streams, one per team member, followed by joint integration, testing and documentation (Figure 3.1). The plan was met; the streams were decoupled through the common interfaces defined in the design phase.

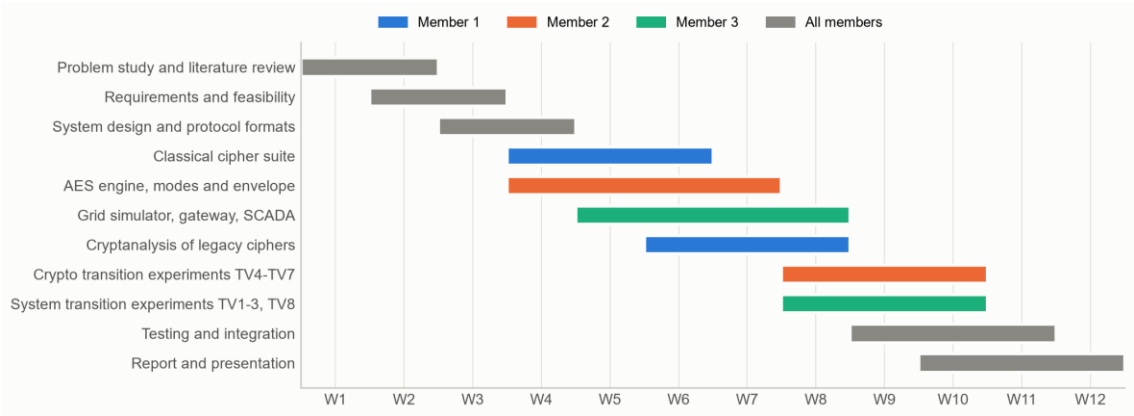



**Figure 3.1:** Project schedule (12 weeks) and allocation of work streams to team members

### 3.6.5 Legal and ethical feasibility

All experiments run against a self-contained simulator; no real utility system, network or data is accessed. The attack code is limited to breaking classical ciphers - long-

documented textbook material - and to demonstrating misconfigurations of the project's own gateway, with every attack paired with a mitigation. The work therefore raises no legal or ethical concerns and its results serve defenders.

## 3.7 Hardware and Software Requirements

**Table 3.5:** Hardware and software requirements

| Item | Requirement | Used in this project |
|---|---|---|
| Processor | Any x86-64 / ARM64 CPU | Intel64 Family 6 Model 140 Stepping 1, GenuineIntel |
| Memory | >= 4 GB RAM | 8 GB+ laptop |
| Operating system | Windows 10/11 or Linux | Windows-10-10.0.26200-SP0 |
| Language | Python >= 3.10 | Python 3.11.9 |
| Cryptography library | pyca/cryptography >= 41 | cryptography 46.0.5 |
| Testing / figures | pytest, matplotlib, numpy | pytest, matplotlib, numpy |
| Documentation build | Node.js 18+, MS Word (PDF export) | docx, pptxgenjs, Word |

# CHAPTER 4: SYSTEM DESIGN AND METHODOLOGY

## 4.1 Methodology

The project follows an iterative *build-attack-mitigate-measure* methodology, adapted from design-science research. Each cycle produces an artefact, subjects it to an adversary, and records quantitative evidence:

1. **Model the legacy system faithfully** - classical ciphers, the legacy frame format and feeder physics - so that attacks and overheads are realistic.
2. **Build the migration artefact** - the secure envelope, the gateway and the migration policies.
3. **Attack** the legacy system and every intermediate configuration a real migration passes through (positions A1 and A2 of the threat model).
4. **Mitigate** each successful attack with a concrete design change implemented in the framework.
5. **Measure** attack success, latency, overhead and data integrity against the objectives of Section 1.4, using fixed random seeds.
6. **Iterate** until every objective is met, then regenerate all reported numbers automatically from the results files.

A key methodological decision is a *single source of truth*: experiments write JSON files, and the figures, this report and the presentation are generated from those files. No number in Chapter 6 is transcribed by hand, which keeps the design, implementation and documentation synchronised.

## 4.2 System Architecture

Figure 4.1 shows the architecture. The **field zone** contains legacy RTUs 1-6, each protected by a different classical cipher; RTU 7, which still sends unencrypted frames and has not yet been migrated; and RTU 8, an upgraded device that seals envelopes itself. The **protocol-translation gateway** sits at the substation boundary and performs the uplink pipeline (legacy decryption, parsing, validation, AES-256-GCM sealing) and the downlink pipeline (command verification and legacy re-encoding). The **control centre** applies the migration policy, verifies envelopes and stores readings. Every box is annotated with the Python module that implements it.

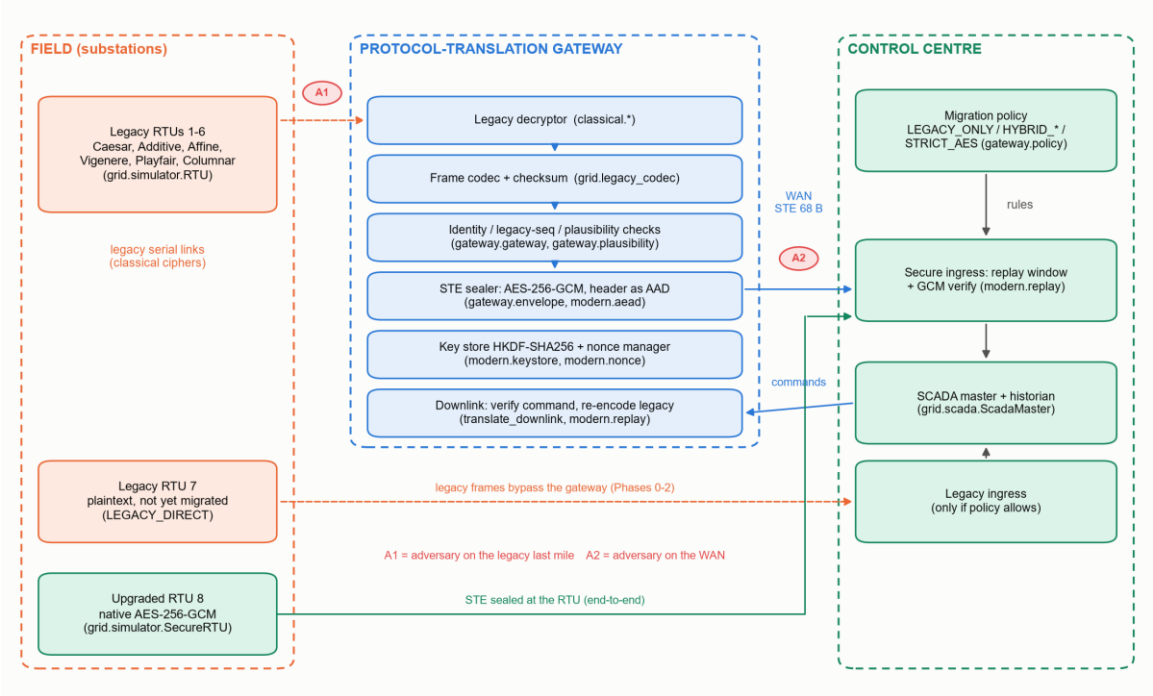



**Figure 4.1:** System architecture of LCMF with adversary positions A1 and A2

Figure 4.2 gives the context-level data-flow diagram (DFD level 0). The system exchanges legacy-encrypted telemetry and commands with legacy RTUs, envelopes with upgraded RTUs, authenticated readings and commands with the SCADA operator, and experiment configuration and results with the security analyst; a master key is provisioned once.

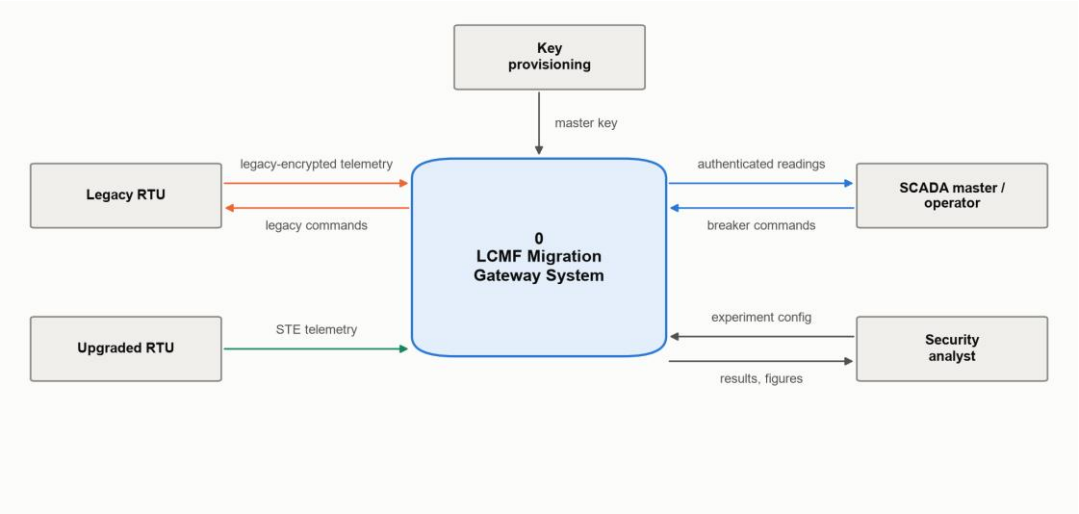



**Figure 4.2:** Data-flow diagram, level 0 (context diagram)

## 4.3 Data Design

### 4.3.1 Legacy telemetry frame

Legacy RTUs send one fixed-width record per poll over the 36-symbol alphabet Z36 = {A-Z, 0-9} (Figure 4.3). The frame is 52 symbols long; 24 symbols are constant tags, which hands an eavesdropper a large amount of known plaintext. A three-digit checksum (the sum of symbol indices modulo 1000) detects line noise but is not cryptographic. The symbol X never occurs in a frame, which allows it to be used as the Playfair filler without ambiguity.

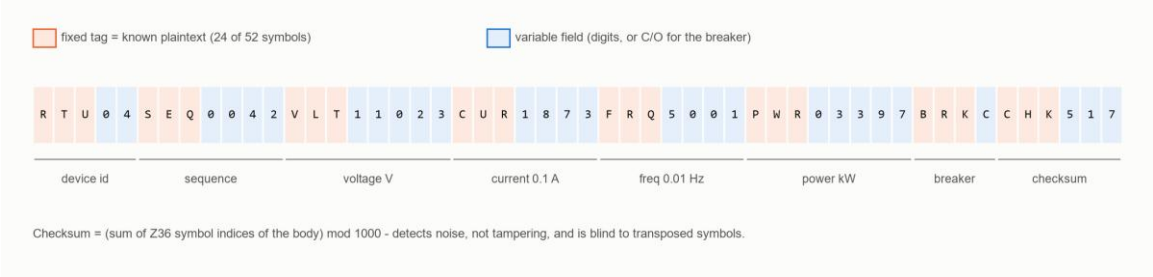



**Figure 4.3:** Legacy Telemetry Frame (LTF): fixed tags (known plaintext) and variable fields

**Table 4.1:** Fields of the legacy telemetry frame

| Tag | Width | Content | Unit / encoding | Example |
|---|---|---|---|---|
| RTU | 2 | Device identifier | decimal | 04 |
| SEQ | 4 | Legacy sequence number | decimal, wraps at 10,000 | 0042 |
| VLT | 5 | Line voltage | volt | 11023 |
| CUR | 4 | Feeder current | 0.1 A | 1873 (187.3 A) |
| FRQ | 4 | System frequency | 0.01 Hz | 5001 (50.01 Hz) |
| PWR | 5 | Active power | kW | 03397 |
| BRK | 1 | Breaker status | C = closed, O = open | C |
| CHK | 3 | Checksum | sum of indices mod 1000 | 517 |

Commands use the frame `RTU04CMDOPENSEQ0007CHKnnn` with the actions OPEN and CLOS.

### 4.3.2 Modern payload

Inside the secure envelope, a reading is serialised as a compact 15-byte big-endian structure (`>IHHIBH`): voltage (4 bytes), current (2), frequency (2), power (4), breaker (1) and the legacy sequence number (2). All quantities are fixed-point integers, so translation between the two formats is exact.

### 4.3.3 Secure Telemetry Envelope (STE)

Figure 4.4 shows the 68-byte envelope. The 37-byte header carries the routing and freshness fields in clear and is passed to GCM as associated data, so that the device identifier, sequence number, timestamp and key generation cannot be altered without invalidating the tag. The overhead over the raw payload is 53 bytes (37-byte header plus 16-byte tag).

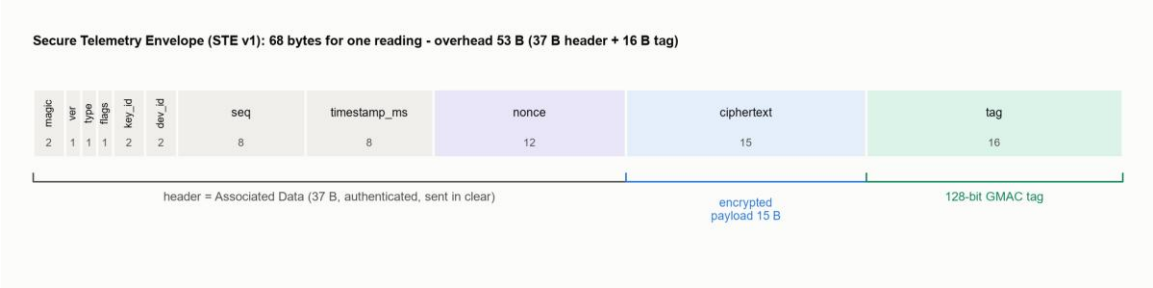



**Figure 4.4:** Byte layout of the Secure Telemetry Envelope (STE v1)

## 4.4 Design of the Legacy Cipher Subsystem

All six ciphers implement one abstract interface, `ClassicalCipher`, with `encrypt`, `decrypt` and `key_space`, and are parameterised by an `Alphabet` object. This lets the same code be verified against textbook vectors over Z26 and then used on telemetry over Z36. Table 4.2 summarises the configured ciphers and their key spaces over Z36.

**Table 4.2:** Legacy ciphers configured in the simulated network

| RTU | Cipher | Key used | Key space over Z36 | Design note |
|---|---|---|---|---|
| 1 | Caesar | shift 3 (fixed) | 1 | Additive cipher with the key frozen into the algorithm |
| 2 | Additive | k = 17 | 36 | C = (P + k) mod 36 |
| 3 | Affine | a = 5, b = 8 | 12 x 36 = 432 | a must be a unit mod 36 |
| 4 | Vigenere | GRIDLOCK | 36^8 (about 2^41) | Key restarts at every frame |
| 5 | Playfair | SUBSTATION | 36! (about 2^138) | 6x6 square, X filler, Q if X is doubled |
| 6 | Columnar | BREAKER | 7! = 5,040 | Irregular (no padding), stable key ranking |
| 7 | None | - | 1 | Unencrypted legacy link |
| 8 | Vigenere / AES | HELIOS, later AES-256-GCM | 36^6 / 2^256 | Upgraded to a native-AES RTU in Phase 1 |

Two design choices deserve explanation. First, the classical 5x5 Playfair merges I and J and cannot represent digits; the project therefore uses a 6x6 square over Z36, which is lossless for telemetry. Second, the columnar transposition is *irregular* - the last row is not padded - so ciphertext length equals plaintext length and decryption is exact.

## 4.5 Design of the Modern Cryptography Engine

AES is implemented from first principles so that its correctness can be inspected and verified (Figure 4.5). The S-box is not copied as a table but *derived*: each byte is inverted in GF(2^8) by computing a^254 and then passed through the affine transformation. MixColumns uses precomputed multiplication tables, and the key schedule supports 128-, 192- and 256-bit keys.

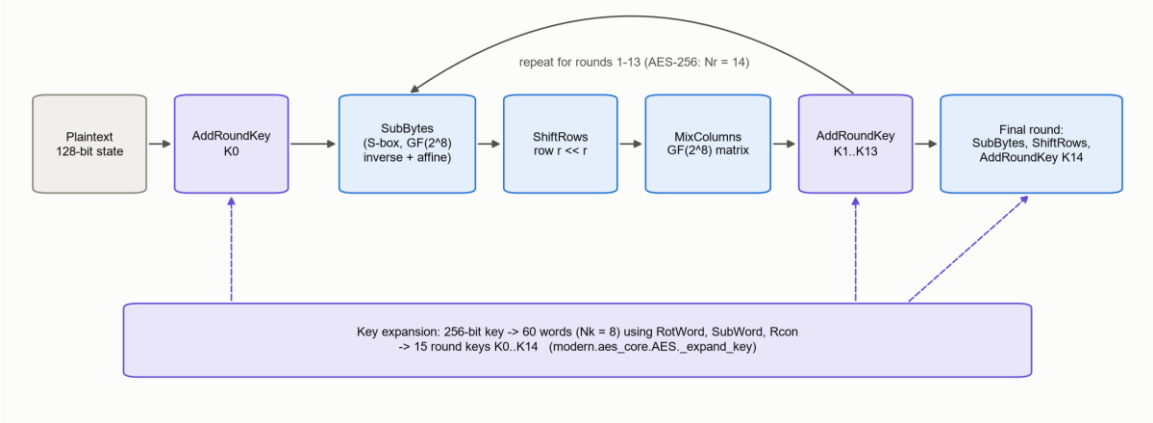



**Figure 4.5:** AES-256 encryption: round structure and key expansion as implemented in aes_core.py

GCM is implemented according to SP 800-38D (Figure 4.6). The GHASH multiplication in GF(2^128) follows Algorithm 1 of the standard, with the reduction constant R = 0xE1 || 0^120. The gateway's production path uses the same mode through the pyca/cryptography library (OpenSSL with AES-NI); both engines expose an identical `seal`/`open` interface and can be swapped by configuration.

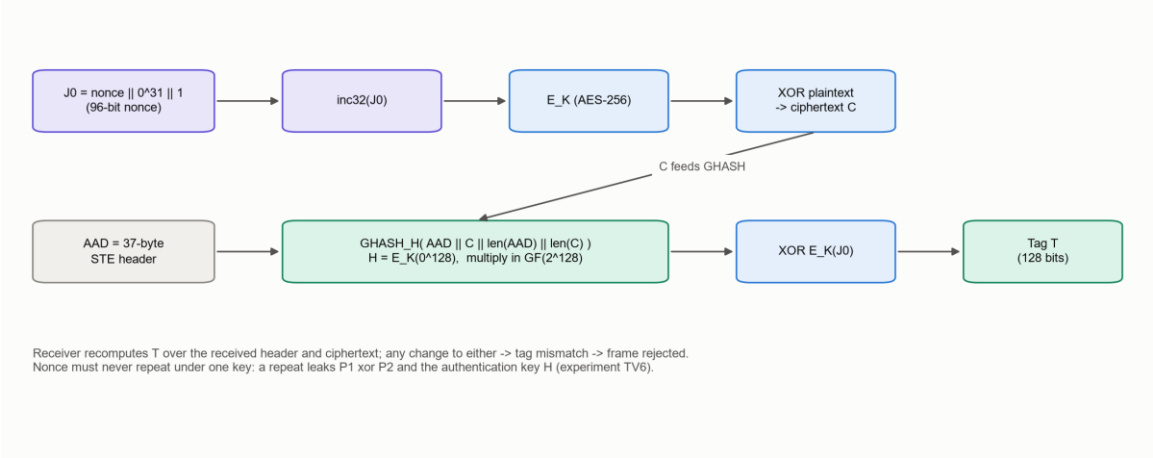



**Figure 4.6:** AES-GCM encryption and tag computation

**Nonce construction.** A GCM nonce must never repeat under a key. The deterministic construction of SP 800-38D Section 8.2.1 is used:

$$\textit{nonce (96 bits)} = \textit{sender_id (32 bits)} \,||\, \textit{message counter (64 bits)} \qquad (4.1)$$

The counter doubles as the envelope sequence number. To survive reboots, the nonce manager reserves counter blocks and writes the high-water mark to disk *before* using any value from the block, so a restarted device resumes above every value it might already have used.

**Key hierarchy.** One 256-bit random master key is provisioned to the gateway and the SCADA master. Working keys are derived per device, per direction and per key generation with HKDF-SHA256 [30]:

$$K(dev, dir, gen) = HKDF\text{-}SHA256(\, IKM = master,\ salt = gen,\ info = \text{"LCMF-v1|dev=<id>|dir=<up|down>"}\,) \qquad (4.2)$$

Compromise of one device key therefore exposes nothing else, and key rotation only increments the generation number carried in the envelope header. **Anti-replay.** The receiver keeps, per device and direction, the highest authenticated sequence number and a 64-bit bitmap of recently seen numbers, as in IPsec ESP [31]; envelopes with timestamps more than 30 s from the receiver clock are also rejected.

## 4.6 Gateway Pipeline and Migration Policy

Figure 4.7 shows the uplink pipeline. A frame is rejected at the first failing stage, and the rejection reason is recorded. The downlink performs the reverse: it checks the envelope's freshness, verifies the tag, decodes the command, re-encodes it as a legacy command frame and encrypts it with the target RTU's legacy cipher.

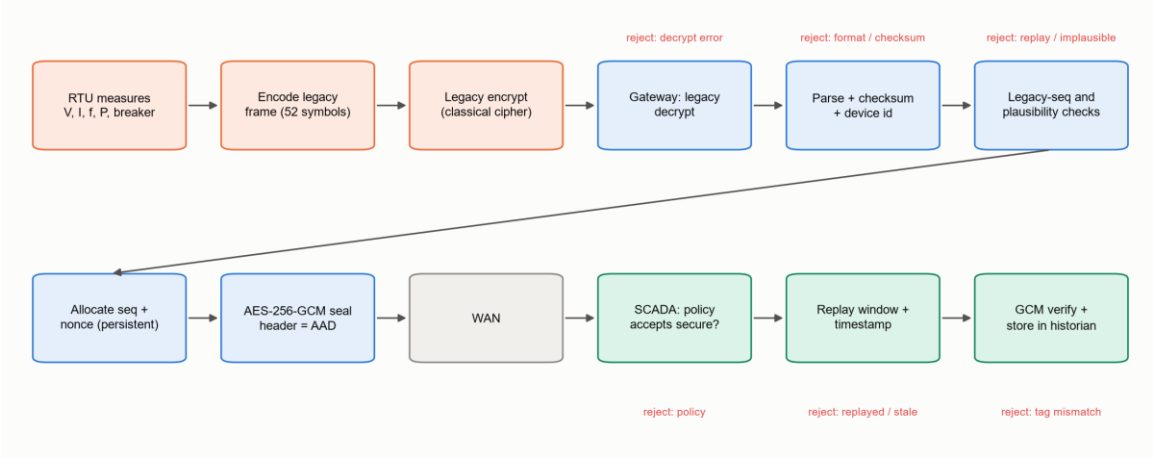



**Figure 4.7:** Uplink processing pipeline from RTU to SCADA historian, with rejection points

A real migration does not switch all devices at once. LCMF models four phases (Figure 4.8); in each phase every device is either *legacy direct* (talks legacy to SCADA), *bridged* (legacy RTU behind the gateway) or *native AES* (upgraded RTU). The control centre enforces the corresponding policy (Table 4.3).

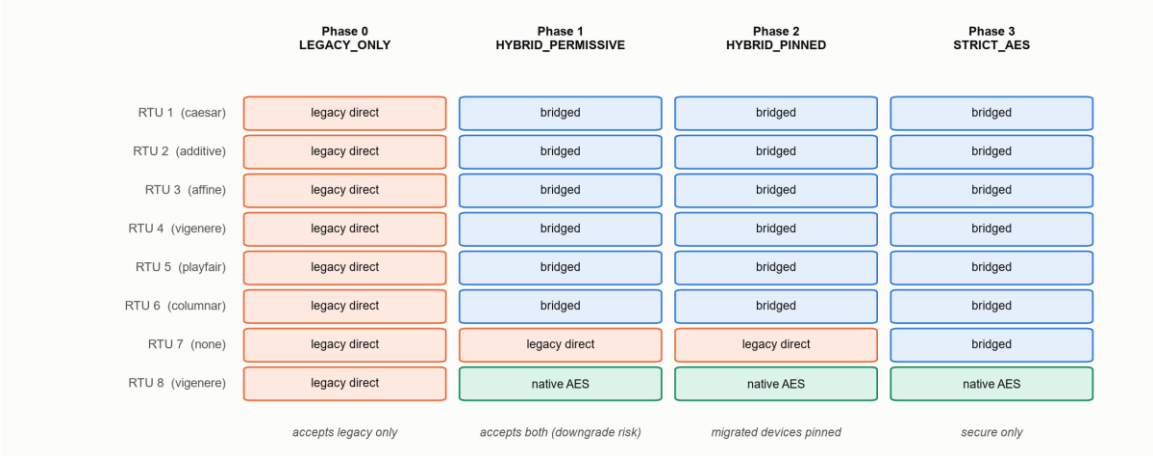



**Figure 4.8:** Migration phases and the state of each device

**Table 4.3:** Migration policies enforced at the control centre

| Mode | Legacy frames accepted from | Secure envelopes | Purpose / risk |
|---|---|---|---|
| LEGACY_ONLY | every device | rejected | Phase 0 baseline |
| HYBRID_PERMISSIVE | every device | accepted | Convenient, but open to downgrade (TV2) |
| HYBRID_PINNED | only devices not yet migrated; a device is pinned once registered or once it sends a valid envelope | accepted | Safe transition (HSTS-like pinning) |
| STRICT_AES | no device | accepted | Phase 3 end state |

## 4.7 Security Analysis and Experiment Design

The attacker's main tool against legacy links is the public frame grammar: any candidate key that decrypts a captured frame into a syntactically valid frame with a correct checksum is almost certainly the right key. Table 4.4 lists the attack designed for each cipher.

**Table 4.4:** Cryptanalysis design per legacy cipher

| Cipher | Attack | Why it works |
|---|---|---|
| Caesar / Additive | Exhaustive search over 36 keys, frame-grammar test | Tiny key space; fixed tags give a perfect plaintext test |
| Affine | Exhaustive search over 432 keys; known-plaintext solution of two congruences | Small key space; linear structure |
| Vigenere | Per-residue constraint propagation: for each key position keep only shifts that map every symbol in that residue class onto the frame template; confirm by checksum | Tags and digit-only fields constrain each key symbol independently |
| Playfair | Keyword dictionary search; known-plaintext digram codebook | Keys are memorable words; digram substitution is deterministic |
| Columnar | Exhaustive search over column orders for up to 7 columns, plus protocol coherence (device id, consecutive sequence numbers) | Small permutation space; a sum checksum cannot detect transpositions |
| English text (textbook) | Chi-squared shift search; Friedman period test; Kasiski examination | Classical statistical attacks for comparison |

Transition vulnerabilities are studied in eight controlled experiments (Table 4.5). Each compares a vulnerable configuration with a mitigated one under an identical adversary and seed.

**Table 4.5:** Design of the transition-vulnerability experiments

| ID | Transition hazard | Adversary and action | Metric | Mitigation tested |
|---|---|---|---|---|
| TV1 | Legacy last mile behind the gateway | A1 breaks the legacy key, then rewrites frames (gross and stealthy false data) | % forged readings accepted by SCADA | Plausibility filter; native AES at the RTU |
| TV2 | Downgrade while both protocols are accepted | A2 strips envelopes of a migrated RTU and injects legacy frames | % forged legacy frames accepted | Policy pinning (HYBRID_PINNED), STRICT_AES |
| TV3 | Replay, including replay laundered by the gateway | A2 replays frames; A1 replays old legacy frames into the gateway | % replays accepted | Sequence window + timestamps; legacy sequence tracking |

| ID | Transition hazard | Adversary and action | Metric | Mitigation tested |
|---|---|---|---|---|
| TV4 | Malleability of legacy and confidentiality-only AES | Blind modification of the breaker field without any key | % successful tampering | AES-GCM authentication |
| TV5 | Deterministic encryption (ECB) of low-entropy status | Passive inference of breaker state from ciphertext equality | Attacker balanced accuracy | Nonce-based AEAD (GCM) |
| TV6 | Nonce reuse after reboot | Passive collection of envelopes across reboots | Nonce collisions; % plaintext recovered | Persistent, reserved counters |
| TV7 | AES keys derived from legacy secrets | Pivot from broken legacy key; dictionary search on one envelope | Key recovered? guesses | HKDF from a random master key |
| TV8 | Overhead on low-bandwidth links | - | Bytes, link time, gateway latency | Compact payload; baud-rate planning |

## 4.8 Module and Class Design

Figure 4.9 shows the package structure and the principal classes. The packages map one-to-one onto the work division between team members, and the only cross-package dependencies are the interfaces `ClassicalCipher`, `AEADEngine`, `TelemetryReading` and the envelope functions.

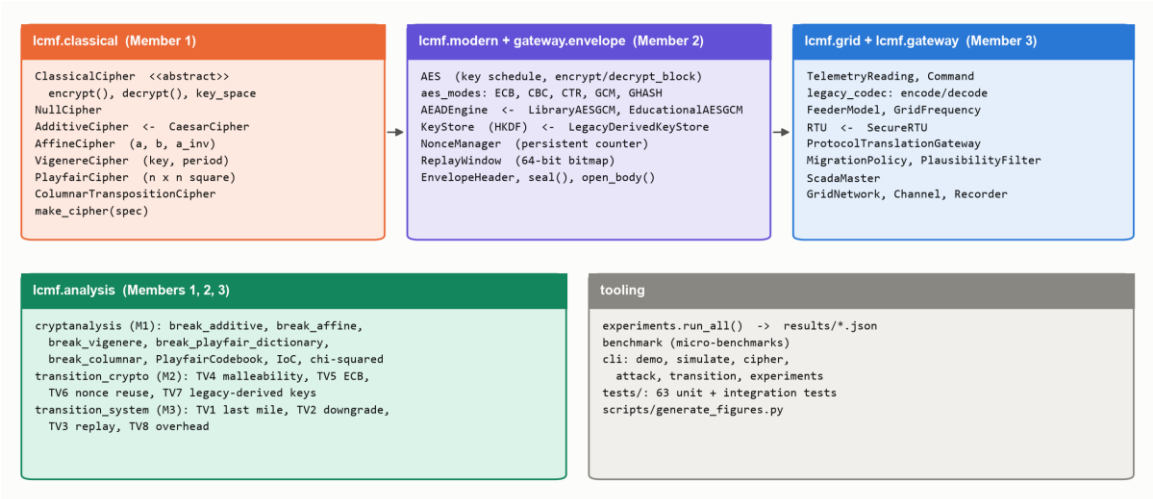



**Figure 4.9:** Package and class design of LCMF (owner of each package in brackets)

## 4.9 Justification of Algorithms and Techniques

**Table 4.6:** Justification of the main algorithms and techniques

| Technique | Why it was chosen | Alternatives considered |
|---|---|---|
| Z36 alphabet for legacy ciphers | Telemetry is numeric; Z36 keeps every cipher lossless while preserving its classical structure | Spelling digits as words (inflates frames); Z26 with digit mapping (ambiguous) |

| Technique | Why it was chosen | Alternatives considered |
|---|---|---|
| 6x6 Playfair | Enciphers digits; filler X never occurs in frames | 5x5 Playfair (cannot represent digits, merges I/J) |
| AES-256 | NIST standard, long security margin, hardware acceleration [20] | AES-128 (sufficient today but smaller margin); ChaCha20-Poly1305 (not a project requirement) |
| GCM mode | Single-pass AEAD; authenticates the header as AAD; widely deployed [25] | CBC + HMAC (two passes, more overhead); CTR or CBC alone (malleable - TV4); ECB (leaks patterns - TV5) |
| Deterministic 96-bit nonces with persistent counters | Uniqueness guaranteed without a random-number generator on the device | Random 96-bit nonces (birthday bound, needs good RNG) |
| HKDF-SHA256 key hierarchy | Key separation per device, direction and generation [30] | One shared key (single point of failure); keys hashed from legacy keywords (TV7) |
| Sliding replay window + timestamps | Tolerates reordering, rejects duplicates and stale frames [31] | Strictly increasing counter only (drops reordered frames) |
| Frame-grammar-based cryptanalysis | Matches the real attacker's knowledge (public protocol formats) | English frequency analysis only (does not apply to numeric telemetry) |
| Physics-based plausibility filter | Cheap defence-in-depth against gross false data on the last mile | Machine-learning anomaly detection (future scope) |

# CHAPTER 5: IMPLEMENTATION

## 5.1 Development Environment and Project Structure

LCMF is implemented in Python 3.11.9. The only runtime dependency is the pyca/cryptography library (46.0.5), used for the production AES-GCM path and HKDF [40]; pytest, matplotlib and numpy are used for testing and figures, and Node.js scripts generate the report and slides. The source tree is shown in Listing 5.1.

**Listing 5.1:** Project directory structure

```
modern project submission/
|-- lcmf/                       Python package (the framework)
|   |-- classical/              legacy cipher suite                    (Member 1)
|   |-- modern/                 AES core, modes, AEAD, keys, nonces    (Member 2)
|   |-- grid/                   telemetry, codec, simulator, SCADA     (Member 3)
|   |-- gateway/                envelope (M2), gateway, policy, filter (M3)
|   |-- analysis/               cryptanalysis (M1), TV experiments (M2, M3), runner
(M2)
|   `-- cli.py                  command-line interface / demonstration
|-- tests/                      unit and integration tests (pytest)
|-- scripts/generate_figures.py  figure pipeline                      (Member 1)
|-- results/                    generated JSON results (single source of truth)
|-- docs/figures/               generated figures
|-- docs/build/                 report and slide generators, team_config.json
|-- requirements.txt, run_demo.bat, build_all.bat, README.md
```

**Table 5.1:** Size of the implementation (non-blank lines)

| Package | Purpose | Lines |
|---|---|---|
| lcmf/classical | Legacy cipher suite | 354 |
| lcmf/modern | AES core, modes, AEAD, keys, nonces, replay | 459 |
| lcmf/grid | Telemetry, codec, simulator, SCADA, network | 563 |
| lcmf/gateway | Envelope, gateway, policy, plausibility | 310 |
| lcmf/analysis | Cryptanalysis, experiments, benchmarks | 1,013 |
| lcmf/cli.py | Command-line interface | 217 |
| tests | Automated tests | 336 |
| scripts | Figure generation | 523 |
| **Total** | | **3,775** |

## 5.2 Legacy Cipher Suite and Cryptanalysis (Member 1)

Each cipher is a small class built on the common interface. The Playfair digram rule (Listing 5.2) is the core of the polygraphic cipher; the same method performs decryption when called with direction -1.

**Listing 5.2:** Playfair digram substitution - lcmf/classical/polyalphabetic.py

```
def _sub(self, a: str, b: str, d: int) -> str:
    n, sq = self.size, self.square
    ra, ca = self.pos[a]
    rb, cb = self.pos[b]
    if ra == rb:                        # same row -> shift along the row
        return sq[ra][(ca + d) % n] + sq[rb][(cb + d) % n]
    if ca == cb:                        # same column -> shift down the column
        return sq[(ra + d) % n][ca] + sq[(rb + d) % n][cb]
    return sq[ra][cb] + sq[rb][ca]    # rectangle -> swap columns
```

The Vigenere attack (Listing 5.3) never guesses the whole key. For each candidate period it computes, independently for every residue class, the set of shifts that map all captured symbols in that class onto symbols allowed by the public frame template. The intersection is usually a single shift, so the key falls out after checking a handful of periods; candidates are confirmed by checksum.

**Listing 5.3:** Template-constrained key search for Vigenere - lcmf/analysis/cryptanalysis.py

```
def  residue candidates(frames: list[str], period: int) -> list[set[int]]:
    A = Z36
    cands = []
    for r in range(period):
        allowed_keys = set(range(36))
        for frame in frames:
            for i in range(r, min(FRAME LEN, len(frame)), period):
                c = A.index(frame[i])
                allowed = TELEMETRY_TEMPLATE[i]
                allowed_keys = {k for k in allowed_keys if A[(c - k) % 36] in allowed}
        cands.append(allowed_keys)
        if not allowed_keys:
            break
    return cands
```

While implementing the columnar attack the team found that the legacy checksum is *transposition-blind*: because it is a plain sum, a wrong column order that swaps two digits still produces a checksum-valid frame (for example `RTU60...SEQ6010` instead of `RTU06...SEQ6001`). The attack therefore adds a protocol-coherence test - one device identifier per link and consecutive sequence numbers - which removes the ambiguity with a second frame.

## 5.3 Modern Cryptography Engine and Secure Envelope (Member 2)

Listing 5.4 shows block encryption in the from-scratch AES. The state is a list of sixteen integers in column-major order, matching FIPS 197.

**Listing 5.4:** AES block encryption - lcmf/modern/aes_core.py

```
def encrypt_block(self, block: bytes) -> bytes:
    if len(block) != 16:
        raise ValueError("AES block must be 16 bytes")
    state = list(block)
    self._add_round_key(state, self.round_keys[0])
    for rnd in range(1, self.nr):
        self._sub_bytes(state, SBOX)
        state = self._shift_rows(state)
        self._mix_columns(state)
        self._add_round_key(state, self.round_keys[rnd])
    self._sub_bytes(state, SBOX)
    state = self._shift_rows(state)
    self._add_round_key(state, self.round_keys[self.nr])
    return bytes(state)
```

GHASH multiplication in GF(2^128) is a direct transcription of Algorithm 1 of SP 800-38D (Listing 5.5); the bit order follows GCM's convention that the most significant bit is the coefficient of x^0.

**Listing 5.5:** Multiplication in GF(2^128) for GHASH - lcmf/modern/aes_modes.py

```
def gf128_mul(x: int, y: int) -> int:
    """Multiplication in GF(2^128) with GCM's bit ordering (SP 800-38D Alg. 1)."""
    z, v = 0, y
    for i in range(127, -1, -1):
        if (x >> i) & 1:
            z ^= v
        v = (v >> 1) ^ _R if v & 1 else v >> 1
    return z
```

The nonce manager (Listing 5.6) implements the crash-safe counter: the high-water mark is persisted before a new block of counters is used.

**Listing 5.6:** Crash-safe nonce allocation - lcmf/modern/nonce.py

```
def next(self) -> tuple[int, bytes]:
    """Return ``(counter, 12-byte nonce)``; the counter doubles as the sequence
number."""
    self._counter += 1
    if self.persist_path and self._counter > self._reserved_until:
        self._reserved_until = self._counter + self.reserve
        self._persist(self._reserved_until)
    if self._counter >= 1 << 64:
        raise OverflowError("nonce counter exhausted - rotate the key")
    return self._counter, self.sender_id.to_bytes(4, "big") +
self._counter.to_bytes(8, "big")
```

## 5.4 Grid Simulator, Gateway and SCADA (Member 3)

The feeder model drives current from a 24-hour load curve and lets the bus voltage sag under load; frequency is a shared mean-reverting random walk around 50 Hz. Every sample satisfies the three-phase power relation, which the plausibility filter later relies on:

$$P = \sqrt{3} \times V \times I \times pf, \quad pf = 0.95 \tag{5.1}$$

Listing 5.7 shows the gateway uplink, which mirrors the pipeline of Figure 4.7 step by step and records per-stage timings.

**Listing 5.7:** Gateway uplink translation - lcmf/gateway/gateway.py

```
def translate_uplink(self, device_id: int, legacy_frame: str, now_ms: int) ->
UplinkResult:
    t0 = time.perf_counter_ns()
    cipher = self.ciphers.get(device_id)
    if cipher is None:
        return self._reject(device_id, "unknown device")
    try:
        plaintext = legacy_decrypt(cipher, legacy_frame)
    except CipherError as exc:
        return self._reject(device_id, f"legacy decrypt failed: {exc}")
    t1 = time.perf_counter_ns()
    try:
        reading = decode_telemetry(plaintext, timestamp_ms=now_ms)
    except FrameError as exc:
        return self._reject(device_id, f"bad legacy frame: {exc}",
plaintext=plaintext)
    if reading.device_id != device_id:
        return self._reject(device_id, "device id mismatch", plaintext=plaintext)
    previous = self.last_reading.get(device_id)
    if self.legacy_seq_check and previous is not None and not
legacy_seq_is_newer(reading.seq, previous.seq):
        return self._reject(device_id, "legacy replay: sequence number not newer",
plaintext=plaintext)
    if self.plausibility is not None:
        ok, issues = self.plausibility.check(reading, previous)
        if not ok:
            return self._reject(device_id, "implausible reading: " + ";
".join(issues), plaintext=plaintext)
    t2 = time.perf counter ns()
    seq, nonce = self.nonce_manager(device_id).next()
    header = ste.EnvelopeHeader(ste.MSG_TELEMETRY, ste.FLAG_LEGACY_ORIGIN,
self.keystore.current_key_id,
                                device_id, seq, now_ms, nonce)
    env = ste.seal(self. engine(device id, UPLINK, header.key id), header,
reading.to_payload())
    t3 = time.perf_counter_ns()
    self.last_reading[device_id] = reading
    self.stats["uplink_ok"] += 1
    timings = {"legacy_decrypt": (t1 - t0) / 1e3, "parse_validate": (t2 - t1) / 1e3,
               "aes seal": (t3 - t2) / 1e3, "total": (t3 - t0) / 1e3}
    return UplinkResult(True, "translated", device_id, plaintext, reading, env,
timings)
```

The migration policy is deliberately small (Listing 5.8); its effect on downgrade attacks is measured in experiment TV2.

**Listing 5.8:** Migration policy decision for legacy frames - lcmf/gateway/policy.py

```
def accepts_legacy(self, device_id: int) -> tuple[bool, str]:
    if self.mode in (MigrationMode.LEGACY_ONLY, MigrationMode.HYBRID_PERMISSIVE):
        return True, "legacy path permitted"
    if self.mode is MigrationMode.HYBRID_PINNED:
        if device_id in self.pinned:
            return False, "device is pinned to secure transport (downgrade blocked)"
        return True, "legacy path permitted for unmigrated device"
    return False, "strict mode: legacy frames rejected"
```

## 5.5 Design-to-Implementation Traceability

Table 5.2 maps every design element of Chapter 4 to the code that implements it, the test that verifies it and the result that evidences it. Every row is complete, so the implementation is fully aligned with the design.

**Table 5.2:** Traceability matrix: requirement - design - implementation - verification - evidence

| Req. | Design (section) | Implementation | Verified by | Evidence |
|---|---|---|---|---|
| FR1 | Cipher suite (4.4) | classical/monoalphabetic.py, polyalphabetic.py, transposition.py | test_classical.py (vectors, round trips) | Table 6.2 |
| FR2 | Legacy frame (4.3.1) | grid/legacy_codec.py | test_gateway.py::test_legacy_frame_format | Figure 4.3 |
| FR3 | Simulator (4.2) | grid/simulator.py, grid/network.py | test_end_to_end_integrity | Table 6.4 |
| FR4 | Uplink pipeline, STE (4.3.3, 4.6) | gateway/gateway.py, gateway/envelope.py | test_gateway_uplink_and_downlink | Table 6.4, Figure 6.4 |
| FR5 | Downlink (4.6) | ProtocolTranslationGateway.translate_downlink | test_breaker_commands_reach_every_kind_of_rtu | Appendix C |
| FR6 | Migration policy (4.6) | gateway/policy.py | test_policy_rules, test_tv2_downgrade_blocked_by_pinning | Table 6.7 |
| FR7 | Anti-replay (4.5) | modern/replay.py, legacy_seq_is_newer | test_replay_window, test_tv3_replay | Table 6.8 |
| FR8 | Plausibility (4.9) | gateway/plausibility.py | test_plausibility_filter, test_tv1_trust_laundering | Table 6.6 |
| FR9 | Cryptanalysis (4.7) | analysis/cryptanalysis.py | six test_break tests | Table 6.3, Figure 6.1 |
| FR10 | TV experiments (4.7) | analysis/transition_system.py, transition_crypto.py | test_tv1 ... test_tv4_to_tv7 | Section 6.5 |
| FR11 | Tooling (4.8) | cli.py, analysis/experiments.py, scripts/ | run_demo.bat (end-to-end) | Appendix A, C |
| NFR1 | Library AEAD engine (4.5) | modern/aead.py (LibraryAESGCM) | benchmark.py, TV8 | Figure 6.4 |
| NFR2 | Compact payload (4.3.2) | TelemetryReading.PAYLOAD (15 B) | test_envelope_layout_and_header_authentication | Table 6.13 |
| NFR4 | AES / GCM / HKDF (4.5) | modern/aes_core.py, aes_modes.py, keystore.py | test_fips197 and test_gcm tests | Table 6.2 |

## 5.6 Testing

Testing combines four techniques: *published test vectors* (textbook cipher examples, FIPS-197 Appendices A-C, GCM specification test cases), *property-based round trips* on randomly generated telemetry frames, *cross-implementation checks* between the educational and library AES-GCM, and *integration tests* that run the whole network in every migration mode and assert that each attack/mitigation pair behaves as designed. The suite contains 63 automated tests, all of which pass (63 passed in 1.25s).

**Table 5.3:** Automated test suites

| Test file | Owner | Focus | Tests |
|---|---|---|---|
| tests/test_classical.py | M1 | Textbook vectors for all six ciphers, key handling, key spaces, telemetry round trips | 20 |
| tests/test_cryptanalysis.py | M1 | Key recovery for every cipher, known-plaintext solvers, textbook attacks, Playfair codebook | 8 |
| tests/test_modern.py | M2 | FIPS-197 and GCM vectors, S-box derivation, engine equivalence, tampering, keys, nonces, replay window, envelope | 16 |
| tests/test_transition_crypto.py | M2 | TV4-TV7: malleability, ECB leakage, nonce reuse, legacy-derived keys | 1 |
| tests/test_gateway.py | M3 | Frame codec, plausibility, gateway uplink/downlink, integrity in all modes, policy, commands | 15 |
| tests/test_transition_system.py | M3 | TV1-TV3: last-mile injection, downgrade, replay - attack and mitigation pairs | 3 |
| **Total** | | | **63** |

## 5.7 Demonstration of the Modules

Every module can be demonstrated live from the command line (Table 5.4). The guided demo walks through all modules in about two minutes and is the basis of the viva demonstration; Appendix C reproduces its output.

**Table 5.4:** Command-line demonstration of each module

| Command | Module demonstrated | What the audience sees |
|---|---|---|
| python -m lcmf demo --interactive | All modules, in order | Legacy ciphertexts, AES vector check, gateway translation with envelope hex, breaker commands, key recovery table, TV1-TV8 summary |
| python -m lcmf cipher affine encrypt TEXT --a 5 --b 8 | Legacy cipher suite (M1) | Encryption/decryption with any cipher and key; key space |

| Command | Module demonstrated | What the audience sees |
|---|---|---|
| python -m lcmf attack | Cryptanalysis (M1) | Every legacy key recovered with effort metrics |
| python -m lcmf simulate --mode STRICT_AES | Simulator, gateway, SCADA (M3) + envelope (M2) | Per-poll readings, path, verdict, wire bytes |
| python -m lcmf transition | Transition experiments (M2, M3) | Raw TV1-TV8 results |
| python -m lcmf experiments | Experiment runner | Objectives table; results/*.json regenerated |
| python -m pytest | All modules | 63 passing tests |

# CHAPTER 6: RESULTS AND DISCUSSION

This chapter reports the measured results of every experiment. All values are produced by `python -m lcmf experiments` and read directly from the `results/` directory; they are reproducible because every experiment uses a fixed random seed.

## 6.1 Experimental Setup

**Table 6.1:** Experimental setup

| Parameter | Value |
|---|---|
| Processor / OS | Intel64 Family 6 Model 140 Stepping 1, GenuineIntel / Windows-10-10.0.26200-SP0 |
| Software | Python 3.11.9, cryptography 46.0.5 |
| Simulated network | 8 feeders of an 11 kV distribution system, 2 s polling cycle, start 08:00 |
| Cryptanalysis data | 10 captured frames per legacy RTU (attacks use as few as needed) |
| Transition experiments | 60 trials (TV1-TV3), 200 frames (TV4), 1,440 one-minute status messages (TV5), 120 polls with a reboot every 30 polls (TV6) |
| Integrity test | 450 polls x 8 devices x 4 modes = 14,400 frames |
| Total run time of the experiment suite | 8.2 s |

## 6.2 Correctness of the Cryptographic Implementations

**Table 6.2:** Validation against published test vectors

| Module | Test vector | Result |
|---|---|---|
| Caesar | ATTACKATDAWN (Z26) | PASS |
| Additive | HELLO, k=15 (Z26) | PASS |
| Affine | AFFINECIPHER, a=5 b=8 | PASS |
| Vigenere | ATTACKATDAWN, LEMON | PASS |
| Playfair | HIDETHEGOLD..., PLAYFAIREXAMPLE | PASS |
| Columnar | WEAREDISCOVERED..., ZEBRAS | PASS |
| AES-128 | FIPS-197 App. C encrypt | PASS |
| AES-128 | FIPS-197 App. C decrypt | PASS |
| AES-192 | FIPS-197 App. C encrypt | PASS |
| AES-192 | FIPS-197 App. C decrypt | PASS |

| Module | Test vector | Result |
|---|---|---|
| AES-256 | FIPS-197 App. C encrypt | PASS |
| AES-256 | FIPS-197 App. C decrypt | PASS |
| AES-128 | FIPS-197 App. B | PASS |
| AES-GCM | GCM spec test case 2 (C\|\|T) | PASS |
| AES-GCM | GCM spec test case 3 (tag) | PASS |

All 15 published vectors pass (Table 6.2). In addition, 70,000 randomly generated telemetry frames were encrypted and decrypted with every legacy cipher with 0 failures, and the educational AES-256-GCM produced output identical to the cryptography library in 500 of 500 random cases with varying message and AAD lengths. Objectives O1 and O2 are therefore met.

## 6.3 Cryptanalysis of the Legacy Ciphers

**Table 6.3:** Key recovery against every legacy cipher from captured traffic

| Cipher | True key | Key space | Attack | Frames | Keys tried | Time (ms) | Recovered |
|---|---|---|---|---|---|---|---|
| Caesar | Additive shift 3 (fixed) | 0 bits | exhaustive key search | 1 | 36 | 0.6 | Yes |
| Additive | k = 17 | 5.2 bits | exhaustive key search | 1 | 36 | 0.6 | Yes |
| Affine | a = 5, b = 8 | 8.8 bits | exhaustive key search | 1 | 432 | 7.5 | Yes |
| Vigenere | GRIDLOCK (L = 8) | 41.4 bits | template constraint propagation + checksum | 1 | 8 | 0.2 | Yes |
| Playfair | SUBSTATION (6x6) | 138.1 bits | keyword dictionary (178 words) | 1 | 178 | 5.0 | Yes |
| Columnar | BREAKER (7 columns) | 12.3 bits | exhaustive permutation search (L<=7) | 2 | 5,912 | 84.7 | Yes |

Every legacy cipher was broken (Table 6.3). The worst case required 2 frames and 85 ms. The result that stands out is that **nominal key space has almost no bearing on security** (Figure 6.1). Playfair has a key space of about 2^138 - larger than AES-128 - yet fell in 5.0 ms because its key was a dictionary word. Vigenere's 2^41 keys were reduced to 8 candidate evaluations because the fixed tags and digit-only fields constrain every key symbol independently.

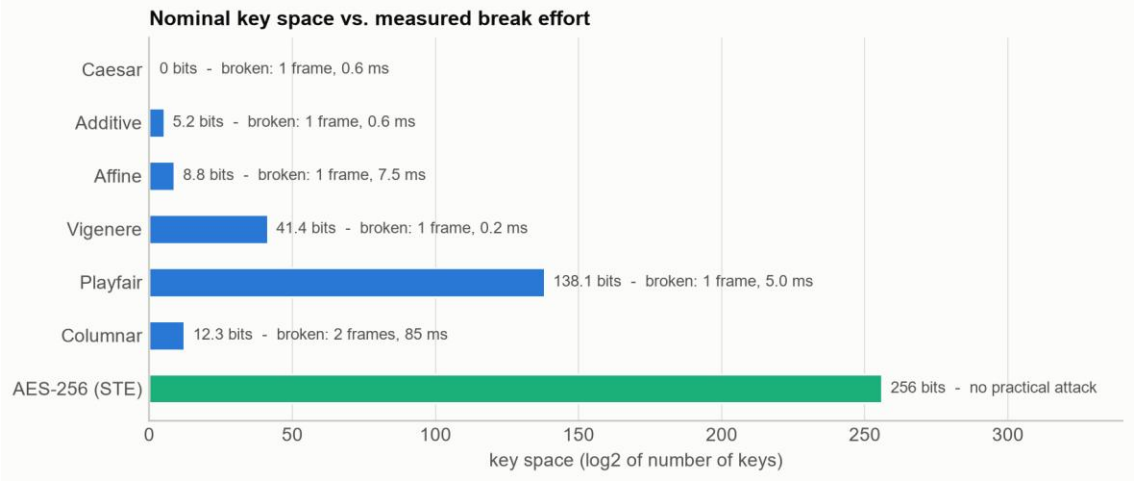



**Figure 6.1:** Nominal key space of each cipher compared with the measured effort to break it

Figure 6.2 explains why. Monoalphabetic ciphers only relabel the symbol distribution: the index of coincidence of Caesar ciphertext (0.0669) is essentially that of the plaintext frames (0.0642), and the transposition preserves it exactly (0.0621). Vigenere (0.0436) and Playfair (0.0476) flatten the profile but remain far from the uniform value 1/36 = 0.028, whereas AES-GCM output is statistically flat.

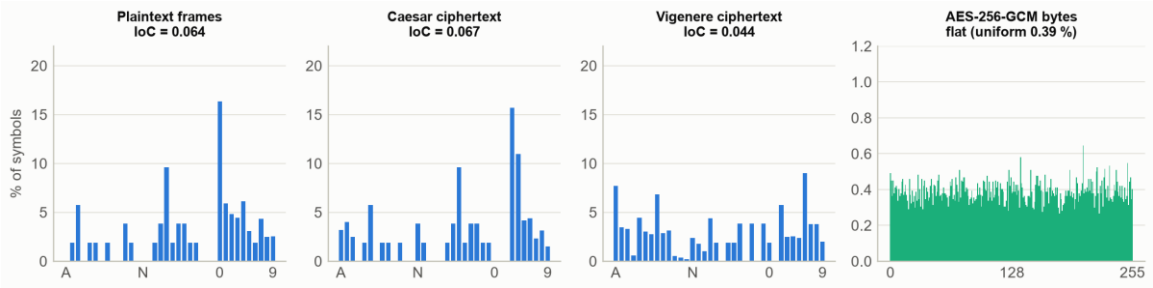



**Figure 6.2:** Symbol frequency profiles and index of coincidence of plaintext and ciphertext

Even without the key, an attacker who knows a few plaintext-ciphertext pairs can decrypt Playfair traffic with a digram codebook (Figure 6.3): one known frame already decrypts 56.3% of the digrams of unseen frames, 90.5% after 20 frames and 100% after 200. Known-plaintext attacks also recovered the Affine key (5, 8) from the three-symbol tag "RTU" alone, and the full Vigenere key "GRIDLOCK" from one frame.

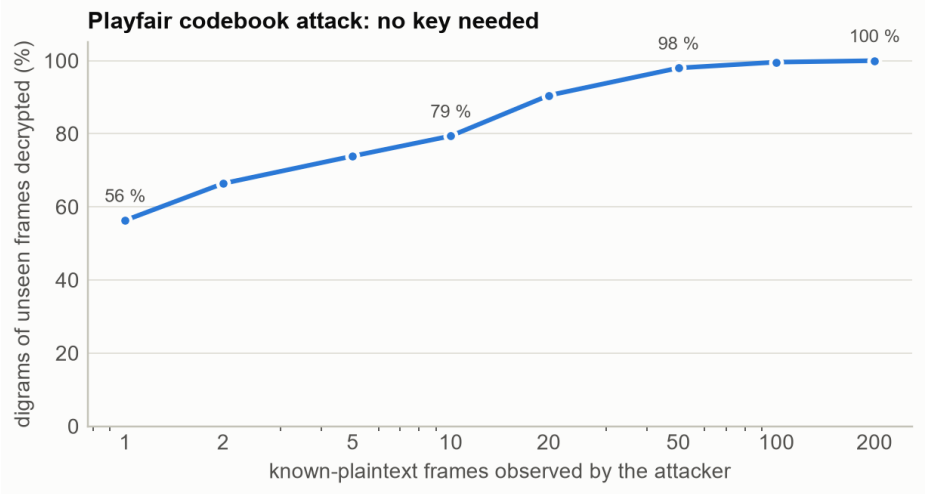



**Figure 6.3:** Playfair digram-codebook attack: decryption coverage versus known frames

For comparison with the textbook setting, the classical statistical attacks were applied to a 700-letter English paragraph. Chi-squared analysis recovered the Caesar shift (3); the Friedman test estimated the Vigenere period as 11, in agreement with the dominant Kasiski factor (11), and per-column chi-squared recovered the key "TRANSFORMER" in 10.34 ms. Telemetry is easier to attack than English because its grammar is exact.

A secondary finding concerns the legacy checksum. Because it is a plain sum of symbol indices, it cannot detect transposed symbols: a wrong columnar key that swaps two digits still yields a checksum-valid frame. Non-cryptographic checksums therefore provide no integrity against an adversary, which is relevant to TV4.

## 6.4 Correctness and Performance of the Gateway

**Table 6.4:** End-to-end delivery and data integrity in each migration mode

| Mode | Frames | Delivered | Corrupted | Via envelope | Via legacy path |
|---|---|---|---|---|---|
| LEGACY_ONLY | 3,600 | 3,600 (100%) | 0 | 0 | 3,600 |
| HYBRID_PERMISSIVE | 3,600 | 3,600 (100%) | 0 | 3,150 | 450 |
| HYBRID_PINNED | 3,600 | 3,600 (100%) | 0 | 3,150 | 450 |
| STRICT_AES | 3,600 | 3,600 (100%) | 0 | 3,600 | 0 |

Across all four modes, 14,400 of 14,400 legitimate frames were delivered and not a single reading was altered in translation (Table 6.4), satisfying the correctness half of objective O3. Figure 6.4 shows the per-frame gateway latency (legacy decryption, parsing, validation and AES-256-GCM sealing). The median is at most 34.9

microseconds for any cipher, about 29 times below the 1 ms target and more than 4 orders of magnitude below the 2 s polling cycle.

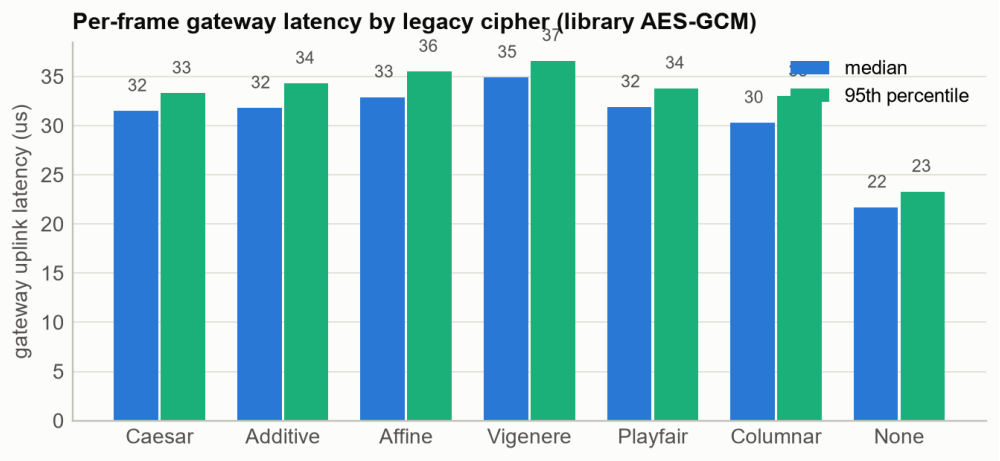



**Figure 6.4:** Gateway uplink latency per legacy cipher with the library AES-GCM engine

**Table 6.5:** Micro-benchmarks (median time per operation)

| Operation | Time (microseconds) |
|---|---|
| Caesar (encrypt / decrypt 52-symbol frame) | 10.8 / 11.0 |
| Additive (encrypt / decrypt 52-symbol frame) | 11.1 / 11.3 |
| Affine (encrypt / decrypt 52-symbol frame) | 11.6 / 11.3 |
| Vigenere (encrypt / decrypt 52-symbol frame) | 13.4 / 14.2 |
| Playfair (encrypt / decrypt 52-symbol frame) | 14.3 / 10.2 |
| Columnar (encrypt / decrypt 52-symbol frame) | 3.1 / 7.8 |
| Plaintext (encrypt / decrypt 52-symbol frame) | 1.6 / 1.6 |
| AES-256 block encrypt (educational) | 72.4 |
| AES-256 key expansion (educational) | 42.7 |
| AES-256-GCM seal 15 B (library) | 1.0 |
| AES-256-GCM open 15 B (library) | 1.0 |
| AES-256-GCM seal 15 B (educational) | 330.3 |
| AES-256-GCM open 15 B (educational) | 304.1 |

The educational AES-GCM, written in pure Python, needs about 330 microseconds to seal a reading against 1.0 microseconds for the library - roughly 330 times slower - and its end-to-end gateway median is 344 microseconds. It would still meet the real-time budget, but the library is used in production because it is constant-time and hardware-accelerated; the educational engine exists for verification and teaching.

## 6.5 Transition Vulnerabilities

This section presents the eight transition vulnerabilities. Each experiment compares a vulnerable configuration with a mitigated one under the same adversary.

### 6.5.1 TV1 - Legacy last mile and trust laundering

A gateway protects only the segment *after* it. An eavesdropper at position A1 recovered the Vigenere key of RTU 4 from 1 frame and could read 100% of the readings that the control centre later received inside AES-GCM envelopes - encryption of the WAN gives no confidentiality for data that is exposed earlier. Worse, the adversary then rewrote frames on the serial link. The gateway faithfully sealed the forged readings into valid envelopes, so SCADA accepted them as authentic (Table 6.6). We call this *trust laundering*: a strong MAC certifies weak data.

**Table 6.6:** TV1 - false-data injection on the legacy last mile

| Defence | Attack | Forged readings accepted |
|---|---|---|
| gateway only | Gross: 9.0 kV under-voltage | 100% |
| gateway only | Stealthy: -3% voltage, power scaled | 100% |
| gateway + plausibility filter | Gross: 9.0 kV under-voltage | 0% |
| gateway + plausibility filter | Stealthy: -3% voltage, power scaled | 100% |
| native AES-GCM at the RTU (end-to-end) | Bit flip on envelope | 0% |

The physics-based plausibility filter blocked every gross forgery but none of the physically consistent ones. Only moving cryptography into the RTU (native AES) closed the attack completely. **Mitigations:** install the gateway as close as possible to the RTU (ideally in the same cabinet), enable plausibility checks, physically protect the last mile, and prioritise RTUs on exposed links (radio, leased lines) for replacement.

### 6.5.2 TV2 - Downgrade attack during hybrid operation

During migration the control centre must still accept legacy frames from devices that have not been migrated. The adversary at A2 suppressed the envelopes of migrated RTU 3 and injected legacy frames reporting its breaker as open, encrypted with the Affine key (5, 8) recovered from traffic recorded before the migration (Table 6.7).

**Table 6.7:** TV2 - downgrade attack against each migration policy

| Policy | Forged legacy frames accepted | Unmigrated device still served? |
|---|---|---|
| HYBRID_PERMISSIVE | 100% | Yes (100% delivered) |
| HYBRID_PINNED | 0% | Yes (100% delivered) |
| STRICT_AES | 0% | No (all devices must be migrated) |

A permissive hybrid policy accepted every downgraded frame. Pinning - remembering that a device has migrated and refusing legacy frames that claim to come from it - stopped the attack completely while still serving the unmigrated RTU 7. STRICT_AES also blocks the attack but is only possible once every device is migrated. **Mitigation:** use HYBRID_PINNED throughout the transition and remove legacy keys from the front end as soon as a device is migrated.

### 6.5.3 TV3 - Replay and freshness laundering

Legacy frames carry no cryptographic freshness, so replayed frames were always accepted on the WAN (Table 6.8). Replayed envelopes were all rejected by the 64-bit sequence window and timestamp check (60 as "replayed sequence number" and 60 as "stale or future timestamp"). The subtle case is replay on the last mile: a *stateless* gateway wraps an old legacy frame into a brand-new envelope with a fresh sequence number, which SCADA cannot distinguish from current data. Tracking the legacy sequence number at the gateway closes this gap.

**Table 6.8:** TV3 - replay attacks

| Scenario | Defence | Replays accepted |
|---|---|---|
| WAN replay of legacy frames | none (legacy) | 100% |
| WAN replay of secure envelopes | 64-bit seq + window + timestamp | 0% |
| Last-mile replay through the gateway | stateless gateway | 100% |
| Last-mile replay through the gateway | legacy sequence tracking at gateway | 0% |

A residual risk remains because the legacy counter has only four digits and wraps after 10,000 frames; at a 2 s poll this is about 5.5 hours, after which very old frames could again look new. **Mitigation:** legacy sequence tracking plus plausibility checks, and native freshness once the RTU is upgraded.

### 6.5.4 TV4 - Malleability: AES alone is not enough

In this experiment the attacker has no key and only wants SCADA to show a closed breaker as open. Against the Vigenere link it suffices to add 12 to the ciphertext symbol of the breaker field (C to O) and patch the checksum digits; this succeeded for 81.5% of frames, failing only when the checksum patch carried into the next digit (Table 6.9). A naive migration to confidentiality-only AES is no better: flipping one ciphertext bit (CTR) or one IV bit (CBC) flips the breaker bit with certainty. Only authenticated encryption detected every modification.

**Table 6.9:** TV4 - blind tampering of the breaker status without the key

| Scheme | Integrity protection | Tampering success |
|---|---|---|
| Legacy Vigenere (+checksum patch) | non-cryptographic checksum | 81.5% |
| AES-256-CBC, no MAC (IV bit flip) | none | 100% |
| AES-256-CTR, no MAC (bit flip) | none | 100% |
| AES-256-GCM (STE) | 128-bit GMAC tag | 0% |

### 6.5.5 TV5 - Pattern leakage with ECB

Status words are low-entropy: a day of 1,440 one-minute status messages from one RTU contains only a handful of distinct plaintexts. Under AES-ECB they produced only 3 distinct ciphertexts, so an attacker who simply labels the most frequent ciphertext as "normal" reconstructed the breaker timeline - including the 62 minutes of maintenance and protection trips - with 100% balanced accuracy (Table 6.10). With GCM every message was unique and the same attacker performed at chance level. Figure 6.5 visualises the effect on a bitmap.

**Table 6.10:** TV5 - inference of breaker state from ciphertext alone

| Mode | Distinct ciphertexts | Attacker balanced accuracy | Precision of outage alarms |
|---|---|---|---|
| AES-256-ECB | 3 | 100% | 100% |
| AES-256-GCM | 1,440 | 50.0% | 4.3% |

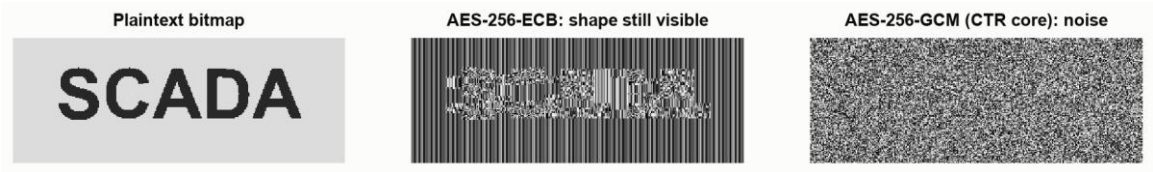



**Figure 6.5:** The same bitmap encrypted with AES-ECB (structure visible) and with GCM's counter mode (noise)

### 6.5.6 TV6 - Nonce reuse after a reboot

Constrained devices often keep counters only in RAM. The gateway was rebooted every 30 polls (Table 6.11). With RAM-only counters it reused 630 nonces; for each collision an observer who knows or predicts one plaintext recovered 100% of the other by XORing the two ciphertexts. The same fault also caused a denial of service: after the first reboot the control centre's replay window rejected 100% of the gateway's envelopes because their sequence numbers looked old. Persistent, block-reserved counters eliminated both problems.

**Table 6.11:** TV6 - GCM nonce management across reboots

| Counter storage | Reboots | Envelopes | Nonce collisions | Plaintext recovered | Delivery after reboot |
|---|---|---|---|---|---|
| RAM only | 3 | 960 | 630 | 100% | 0% |
| persistent (reserved blocks) | 3 | 960 | 0 | 0% | 100% |

Nonce reuse in GCM also exposes the authentication subkey H, enabling forgeries [28], [29]; that attack is not reproduced here but motivates treating any collision as a key-compromise event.

### 6.5.7 TV7 - AES keys derived from legacy secrets

A tempting shortcut during migration is to reuse existing secrets, for example by hashing the old Vigenere keyword into an AES key. Table 6.12 shows the consequence. An attacker who had already broken the legacy cipher (one frame, Section 6.3) derived the AES key directly. Even without legacy traffic, a 178-word dictionary found the key "GRIDLOCK" after 101 guesses in 0.48 ms by testing each candidate against the GCM tag of a single envelope. The effective strength of such an AES-256 key is about 7.5 bits.

**Table 6.12:** TV7 - key provenance

| Key derivation | Pivot from legacy key | Dictionary attack | Effective strength |
|---|---|---|---|
| SHA-256(legacy keyword) | Succeeds | Found "GRIDLOCK" after 101 guesses | 7.5 bits |
| HKDF-SHA256(random 256-bit master) | Fails | Fails (178 guesses) | 256 bits |

### 6.5.8 TV8 - Bandwidth and latency overhead

The envelope enlarges a reading from 52 legacy bytes to 68 bytes (+30.8%). Figure 6.6 and Table 6.13 translate this into serial-link time for a scan of all eight RTUs (10 bits per byte, 8N1 framing).

**Table 6.13:** TV8 - link time per frame and per 8-RTU scan

| Baud rate | Legacy frame (ms) | Envelope (ms) | 8-RTU scan, legacy (ms) | 8-RTU scan, envelope (ms) |
|---|---|---|---|---|
| 1,200 | 433.3 | 566.7 | 3467 | 4533 |
| 9,600 | 54.2 | 70.8 | 433 | 567 |
| 19,200 | 27.1 | 35.4 | 217 | 283 |
| 115,200 | 4.5 | 5.9 | 36 | 47 |

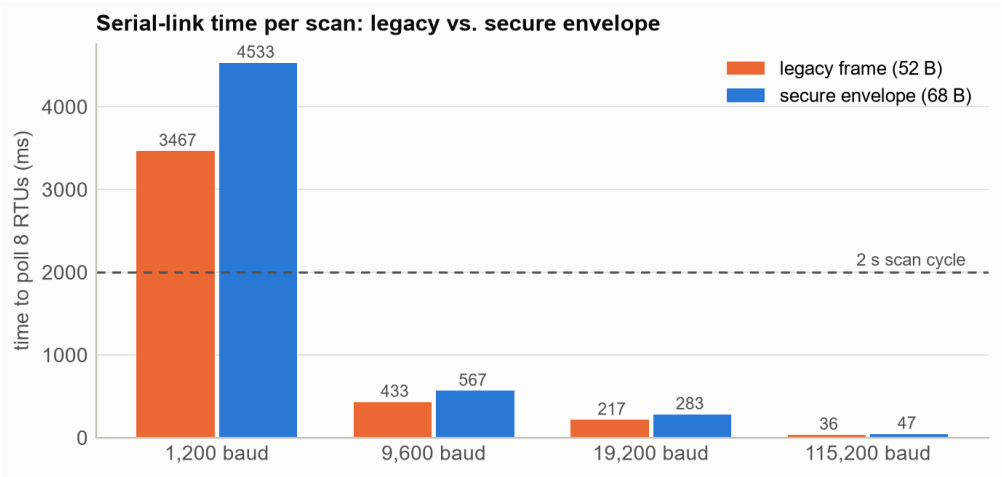



**Figure 6.6:** Serial-link time to poll eight RTUs, legacy frames versus secure envelopes

At 9,600 baud and above the envelope fits comfortably within a 2 s scan. At 1,200 baud even the legacy protocol already overruns the cycle for eight RTUs, and the envelope lengthens the scan by a further ${t8.size_increase_pct}%; such links need fewer RTUs per line, a longer scan cycle or a bandwidth upgrade. Since the envelope is added on

the gateway-to-control-centre segment, which is normally IP, the serial overhead matters chiefly for upgraded RTUs that still use slow radio links.

Figure 6.7 summarises the attack success rates of the vulnerable and mitigated configurations for TV1-TV7.

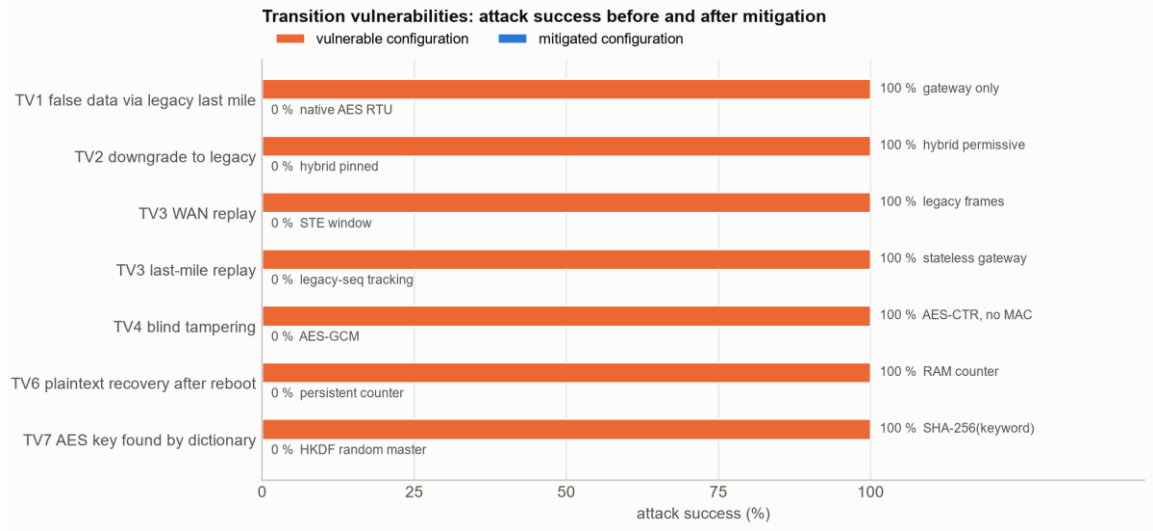



**Figure 6.7:** Attack success before and after mitigation for each transition vulnerability

## 6.6 Risk and Mitigation Matrix

**Table 6.14:** Risk assessment of the transition vulnerabilities (L = low, M = medium, H = high)

| ID | Vulnerability | Likelihood | Impact | Risk | Mitigation implemented in LCMF | Residual risk |
|---|---|---|---|---|---|---|
| TV1 | Legacy last mile behind gateway | H | H | High | Plausibility filter; native-AES RTU | Stealthy false data until RTU upgraded |
| TV2 | Downgrade in hybrid mode | M | H | High | HYBRID_PINNED policy; STRICT_AES | None once pinned |
| TV3 | Replay / freshness laundering | H | M | High | 64-bit window + timestamps; legacy sequence tracking | Legacy counter wrap (about 5.5 h) |
| TV4 | Malleable encryption | M | H | High | AES-256-GCM with header as AAD | None |
| TV5 | ECB pattern leakage | M | M | Medium | Nonce-based AEAD only | Traffic-volume analysis |
| TV6 | Nonce reuse after reboot | M | H | High | Persistent, reserved counters | Storage failure; monitor collisions |
| TV7 | Keys derived from legacy secrets | M | H | High | HKDF from random 256-bit master; key rotation | Master-key protection (HSM: future) |
| TV8 | Bandwidth overhead | L | M | Low | Compact 15-byte payload | 1,200-baud links need planning |

## 6.7 Achievement of Objectives

**Table 6.15:** Objectives, targets and measured results

| ID | Target | Measured result | Status |
|---|---|---|---|
| O1 | 100 % of textbook vectors; 0 round-trip failures on 10,000 frames per cipher | 6/6 vectors; 0 failures in 70,000 round trips | Met |
| O2 | 100 % of FIPS-197 / GCM vectors; bit-identical to library on 500 cases | 9/9 vectors; 500/500 identical | Met |
| O3 | 0 corrupted readings in all 4 modes; median gateway latency < 1 ms | 0 corrupted of 14,400; worst median 34.9 us | Met |
| O4 | key recovered from <= 3 frames in < 1 s for each cipher | 6/6 keys; max 2 frames; max 85 ms | Met |
| O5 | 0 % acceptance in every STRICT / PINNED / GCM scenario | max acceptance 0.0 % across 6 scenarios | Met |
| O6 | >= 8 vulnerabilities, each with a measured mitigation | 8 vulnerabilities (TV1-TV8) measured | Met |
| O7 | overhead <= 64 B/frame; 8-device scan at 9600 baud < 2 s cycle | 53 B overhead; scan 567 ms | Met |

All 7 objectives were met (Table 6.15).

## 6.8 Discussion

Five lessons emerge from the results.

1. **Structure, not key length, defeats classical ciphers.** Every legacy cipher fell in milliseconds because telemetry frames are rigidly formatted; the 2^138 key space of Playfair was irrelevant.
2. **A gateway moves the trust boundary but does not remove the legacy link.** The last mile remains exactly as weak as before (TV1, TV3), and the gateway can even launder forged or replayed data into authentic-looking envelopes. Defence-in-depth (plausibility, sequence tracking) helps, but only native cryptography in the RTU closes the gap.
3. **Choosing AES is necessary but far from sufficient.** Mode (TV4, TV5), nonce management (TV6) and key provenance (TV7) each turned AES-256 into something weaker than intended. Authenticated encryption, persistent counters and random master keys are essential.
4. **The migration policy is a security control.** Accepting both protocols without pinning re-opens every legacy weakness through downgrade (TV2).

5. **The overhead is affordable.** The envelope costs 53 bytes and tens of microseconds per frame; only 1,200-baud links need re-planning (TV8).

Based on these results, we recommend the following migration playbook: (1) inventory legacy links and rank them by exposure; (2) deploy gateways with plausibility checks and legacy sequence tracking, as close to the RTUs as possible; (3) operate the control centre in HYBRID_PINNED mode, registering each device as migrated the moment its gateway is commissioned; (4) provision fresh random master keys - never derive keys from legacy secrets - and use persistent nonce counters; (5) replace RTUs on the most exposed links with native-AES devices first; and (6) switch to STRICT_AES and delete legacy keys once the last device is migrated.

# CHAPTER 7: CONCLUSION AND FUTURE SCOPE

## 7.1 Conclusion

This project designed, implemented and evaluated LCMF, a Legacy Cipher Migration Framework for industrial control systems. It reproduces a smart-grid environment protected by six classical ciphers, implements AES and GCM from first principles alongside a production library path, and provides a protocol-translation gateway that wraps legacy telemetry into a 68-byte AES-256-GCM envelope with authenticated headers, anti-replay protection and four migration policies. The gateway translated 14,400 frames without a single error at a median latency of at most 34.9 microseconds.

The evaluation confirms that classical ciphers provide no protection for structured telemetry - every key was recovered from at most two frames - and, more importantly, that the transition to AES introduces risks of its own. Eight transition vulnerabilities were demonstrated and quantified, and for each one a mitigation implemented in the framework reduced the attack success to zero, except for physically consistent false data on the legacy last mile, which only replacing the RTU removes. The resulting migration playbook gives utilities a concrete, evidence-based sequence for upgrading legacy grid hardware safely.

## 7.2 Limitations

- The environment is simulated; real Modbus, DNP3 or IEC 60870-5-104 framing and timing are abstracted into the legacy telemetry frame.
- The gateway and control centre hosts are assumed to be uncompromised; key storage uses files rather than hardware security modules.
- The Playfair attacks assume dictionary keywords or known plaintext; a ciphertext-only attack on random squares (for example by hill climbing) was not implemented.
- The plausibility filter uses fixed engineering limits and cannot detect physically consistent false data.

## 7.3 Future Scope

- Deploy the gateway on embedded hardware (for example an industrial single-board computer) and measure latency on real serial and radio links.

- Add adapters for Modbus RTU and DNP3 and interoperate with IEC 62351 / TLS on the control-centre side.
- Store master keys in a hardware security module or TPM and automate key rotation.
- Replace the rule-based plausibility filter with state estimation or machine-learning anomaly detection to address stealthy false-data injection.
- Add post-quantum or hybrid key establishment for long-lived field devices.
- Formally verify the migration-policy state machine and the envelope parser.

# REFERENCES


[1] K. Stouffer, M. Pease, C. Tang, T. Zimmerman, V. Pillitteri, S. Lightman, A. Hahn, S. Saravia, A. Sherule, and M. Thompson, "Guide to Operational Technology (OT) Security," NIST Special Publication 800-82 Rev. 3, National Institute of Standards and Technology, Gaithersburg, MD, USA, Sep. 2023.

[2] V. C. Gungor, D. Sahin, T. Kocak, S. Ergut, C. Buccella, C. Cecati, and G. P. Hancke, "Smart grid technologies: Communication technologies and standards," IEEE Transactions on Industrial Informatics, vol. 7, no. 4, pp. 529-539, Nov. 2011.

[3] V. M. Igure, S. A. Laughter, and R. D. Williams, "Security issues in SCADA networks," Computers & Security, vol. 25, no. 7, pp. 498-506, Oct. 2006.

[4] E. D. Knapp and J. T. Langill, Industrial Network Security: Securing Critical Infrastructure Networks for Smart Grid, SCADA, and Other Industrial Control Systems, 2nd ed. Waltham, MA, USA: Syngress, 2014.

[5] The Smart Grid Interoperability Panel - Smart Grid Cybersecurity Committee, "Guidelines for Smart Grid Cybersecurity," NIST Interagency Report 7628 Rev. 1, National Institute of Standards and Technology, Gaithersburg, MD, USA, Sep. 2014.

[6] W. Wang and Z. Lu, "Cyber security in the smart grid: Survey and challenges," Computer Networks, vol. 57, no. 5, pp. 1344-1371, Apr. 2013.

[7] American Gas Association, "Cryptographic Protection of SCADA Communications, Part 1: Background, Policies and Test Plan," AGA Report No. 12, Part 1, Washington, DC, USA, 2006.

[8] IEEE Standard for Secure SCADA Communications Protocol (SSCP), IEEE Std 1711.2-2019.

[9] R. M. Lee, M. J. Assante, and T. Conway, "Analysis of the cyber attack on the Ukrainian power grid," SANS Industrial Control Systems and Electricity Information Sharing and Analysis Center (E-ISAC), Defense Use Case, Mar. 2016.

[10] R. Langner, "Stuxnet: Dissecting a cyberwarfare weapon," IEEE Security & Privacy, vol. 9, no. 3, pp. 49-51, May/Jun. 2011.

[11] Y. Liu, P. Ning, and M. K. Reiter, "False data injection attacks against state estimation in electric power grids," in Proc. 16th ACM Conference on Computer and Communications Security (CCS), Chicago, IL, USA, 2009, pp. 21-32.

[12] Y. Mo, T. H.-J. Kim, K. Brancik, D. Dickinson, H. Lee, A. Perrig, and B. Sinopoli, "Cyber-physical security of a smart grid infrastructure," Proceedings of the IEEE, vol. 100, no. 1, pp. 195-209, Jan. 2012.

[13] W. Stallings, Cryptography and Network Security: Principles and Practice, 7th ed. Harlow, U.K.: Pearson, 2017.

[14] B. A. Forouzan and D. Mukhopadhyay, Cryptography and Network Security, 2nd ed. New Delhi, India: Tata McGraw-Hill, 2010.

[15] D. Kahn, The Codebreakers: The Story of Secret Writing. New York, NY, USA: Macmillan, 1967.

[16] A. Kerckhoffs, "La cryptographie militaire," Journal des sciences militaires, vol. IX, pp. 5-38, Jan. 1883.

[17] A. Sinkov, Elementary Cryptanalysis: A Mathematical Approach. Washington, DC, USA: Mathematical Association of America, 1966.

[18] F. W. Kasiski, Die Geheimschriften und die Dechiffrir-Kunst. Berlin, Germany: E. S. Mittler und Sohn, 1863.

[19] W. F. Friedman, The Index of Coincidence and Its Applications in Cryptography, Publication No. 22. Geneva, IL, USA: Riverbank Laboratories, 1922.

[20] National Institute of Standards and Technology, "Advanced Encryption Standard (AES)," Federal Information Processing Standards Publication 197, Nov. 2001 (updated May 2023).

[21] J. Daemen and V. Rijmen, The Design of Rijndael: AES - The Advanced Encryption Standard. Berlin, Germany: Springer, 2002.

[22] J. Katz and Y. Lindell, Introduction to Modern Cryptography, 3rd ed. Boca Raton, FL, USA: CRC Press, 2020.

[23] M. Dworkin, "Recommendation for Block Cipher Modes of Operation: Methods and Techniques," NIST Special Publication 800-38A, Dec. 2001.

[24] S. Vaudenay, "Security flaws induced by CBC padding - Applications to SSL, IPSEC, WTLS ...," in Advances in Cryptology - EUROCRYPT 2002, Lecture Notes in Computer Science, vol. 2332. Berlin, Germany: Springer, 2002, pp. 534-545.

[25] M. Dworkin, "Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC," NIST Special Publication 800-38D, Nov. 2007.

[26] D. A. McGrew and J. Viega, "The security and performance of the Galois/Counter Mode (GCM) of operation," in Progress in Cryptology - INDOCRYPT 2004, Lecture Notes in Computer Science, vol. 3348. Berlin, Germany: Springer, 2004, pp. 343-355.

[27] N. Ferguson, B. Schneier, and T. Kohno, Cryptography Engineering: Design Principles and Practical Applications. Indianapolis, IN, USA: Wiley, 2010.

[28] A. Joux, "Authentication failures in NIST version of GCM," public comment submitted to NIST on the draft of SP 800-38D, 2006.

[29] H. Bock, A. Zauner, S. Devlin, J. Somorovsky, and P. Jovanovic, "Nonce-disrespecting adversaries: Practical forgery attacks on GCM in TLS," in Proc. 10th USENIX Workshop on Offensive Technologies (WOOT 16), Austin, TX, USA, 2016.

[30] H. Krawczyk and P. Eronen, "HMAC-based Extract-and-Expand Key Derivation Function (HKDF)," IETF RFC 5869, May 2010.

[31] S. Kent, "IP Encapsulating Security Payload (ESP)," IETF RFC 4303, Dec. 2005.

[32] IEEE Standard for Electric Power Systems Communications - Distributed Network Protocol (DNP3), IEEE Std 1815-2012, 2012.

[33] G. Gilchrist, "Secure authentication for DNP3," in Proc. IEEE Power and Energy Society General Meeting, Pittsburgh, PA, USA, 2008.

[34] IEC 62351 (all parts), Power Systems Management and Associated Information Exchange - Data and Communications Security. Geneva, Switzerland: International Electrotechnical Commission.

[35] Modbus Organization, "MODBUS/TCP Security Protocol Specification," Modbus.org, 2018.

[36] IEC 62443 (all parts), Security for Industrial Automation and Control Systems. Geneva, Switzerland: International Electrotechnical Commission.

[37] A. K. Wright, J. A. Kinast, and J. McCarty, "Low-latency cryptographic protection for SCADA communications," in Applied Cryptography and Network Security (ACNS 2004), Lecture Notes in Computer Science, vol. 3089. Berlin, Germany: Springer, 2004.

[38] P. P. Tsang and S. W. Smith, "YASIR: A low-latency, high-integrity security retrofit for legacy SCADA systems," in Proc. IFIP TC 11 23rd International Information Security Conference (SEC 2008). Boston, MA, USA: Springer, 2008.

[39] I. N. Fovino, A. Carcano, M. Masera, and A. Trombetta, "Design and implementation of a secure Modbus protocol," in Critical Infrastructure Protection III, IFIP Advances in Information and Communication Technology, vol. 311. Berlin, Germany: Springer, 2009.

[40] Python Cryptographic Authority, "cryptography" library documentation. [Online]. Available: https://cryptography.io (accessed Oct. 2026).

# APPENDIX A: USER MANUAL

**Installation.** Install Python 3.10 or later, open a terminal in the project folder and run `pip install -r requirements.txt`. On Windows, `run_demo.bat` performs the installation, runs the test suite, regenerates the results and starts the guided demonstration.

**Table A.1:** Commands provided by the framework

| Command | Purpose |
|---|---|
| python -m lcmf demo [--interactive] | Guided end-to-end demonstration of all modules |
| python -m lcmf simulate --mode MODE --polls N [--engine educational] | Run the grid simulation in a migration mode |
| python -m lcmf cipher TYPE encrypt\|decrypt TEXT [--key K] [--a A --b B] [--z26] | Use any legacy cipher |
| python -m lcmf attack | Recover the key of every legacy cipher |
| python -m lcmf transition | Run TV1-TV8 and print raw results |
| python -m lcmf experiments | Regenerate results/*.json and print the objectives table |
| python -m pytest | Run the automated test suite |
| python scripts/generate_figures.py | Regenerate docs/figures/*.png |
| build_all.bat | Regenerate results, figures, report (DOCX/PDF) and slides |

**Updating team details.** Edit `docs/build/team_config.json` (names, enrolment numbers, course and faculty) and run `build_all.bat`; the cover page, certificate, declaration, contribution appendix and slides are regenerated with the new details.

# APPENDIX B: INDIVIDUAL CONTRIBUTIONS

The work was divided into three modules of comparable size, each owned end-to-end - design, code, tests, report sections and presentation slides - by one member. Problem analysis, integration, testing and documentation were shared. Table B.1 summarises the division and Table B.2 the responsibility for each task.

**Table B.1:** Contribution of each team member

| Member | Module owned | Main contributions |
|---|---|---|
| [Student Name 1] [Enrolment No. 1] | Legacy Cipher Suite, Cryptanalysis & Result Visualisation | Designed the Z36 alphabet abstraction and the common ClassicalCipher interface; Implemented Caesar, Additive, Affine, Vigenere, Playfair (5x5 and 6x6) and Columnar transposition; Verified all ciphers against published textbook vectors and 70,000 telemetry round trips; Built the cryptanalysis module: exhaustive search, template-constrained Vigenere attack, Playfair dictionary and codebook attacks, Friedman/Kasiski/chi-squared tools; Discovered and documented the transposition-blind legacy checksum; Built the figure pipeline that turns results/*.json into every chart and diagram of the report and slides. |
| [Student Name 2] [Enrolment No. 2] | Modern Cryptography Engine, Secure Envelope & Experiment Validation | Implemented AES-128/192/256 from first principles (GF(2^8) arithmetic, derived S-box, key expansion); Implemented ECB, CBC, CTR and GCM (GHASH in GF(2^128)); cross-validated against the cryptography library; Designed the 68-byte Secure Telemetry Envelope with header-as-AAD; Implemented HKDF key store, persistent nonce manager and anti-replay window; Designed and ran crypto transition experiments TV4-TV7; Built the experiment runner, published-vector validation, benchmarks and the objectives evaluation (results/*.json). |
| [Student Name 3] [Enrolment No. 3] | Smart-Grid Simulation, Gateway & System Evaluation | Built the 11 kV feeder model, legacy RTUs, upgraded RTU, SCADA master and network simulator; Implemented the bidirectional protocol-translation gateway and the four migration modes; Implemented the plausibility filter and legacy-sequence tracking mitigations; Designed and ran system transition experiments TV1-TV3 and TV8; Built the command-line interface and the guided viva demonstration. |

**Table B.2:** Task responsibility matrix

| Task | Member 1 | Member 2 | Member 3 |
|---|---|---|---|
| Problem analysis, literature review, feasibility | Lead | Lead | Lead |
| Legacy cipher suite (6 ciphers, Z36 alphabet) | Lead | Support | - |
| Cryptanalysis of legacy ciphers | Lead | - | Support |
| AES core, modes of operation, GCM | - | Lead | - |
| Secure envelope, keys, nonces, replay window | Support | Lead | Support |
| Grid simulator, RTUs, SCADA master | - | - | Lead |
| Translation gateway, migration policy, plausibility | Support | Support | Lead |

| Task | Member 1 | Member 2 | Member 3 |
|---|---|---|---|
| Transition experiments TV4-TV7 | - | Lead | Support |
| Transition experiments TV1-TV3, TV8 | Support | - | Lead |
| Experiment runner, vector validation, benchmarks | - | Lead | Support |
| Result visualisation (figures and diagrams) | Lead | Support | - |
| Command-line interface and viva demonstration | Support | - | Lead |
| Testing and integration | Lead | Lead | Lead |
| Report and presentation | Lead | Lead | Lead |

**Table B.3:** Code, report and presentation ownership

| Member | Code owned | Lines of code | Report sections | Slides |
|---|---|---|---|---|
| [Student Name 1] | lcmf/classical/*, lcmf/analysis/cryptanalysis.py, scripts/generate_figures.py, tests/test_classical.py, tests/test_cryptanalysis.py | 1,269 | Chapter 2 (sections 2.1-2.4: classical ciphers and their cryptanalysis); Section 4.4 (legacy cipher design); Section 5.2; Section 6.3 | Slides 6-9 (legacy ciphers and how they fall) |
| [Student Name 2] | lcmf/modern/*, lcmf/gateway/envelope.py, lcmf/analysis/transition_crypto.py, lcmf/analysis/experiments.py, lcmf/analysis/benchmark.py, tests/test_modern.py, tests/test_transition_crypto.py | 1,184 | Chapter 2 (sections 2.5-2.6: AES and authenticated encryption); Section 4.5 (secure envelope and key management); Sections 5.3, 5.6; Sections 6.1, 6.2, 6.5.4-6.5.7 | Slides 10-13 (AES engine, envelope, crypto pitfalls) |
| [Student Name 3] | lcmf/grid/*, lcmf/gateway/gateway.py, lcmf/gateway/policy.py, lcmf/gateway/plausibility.py, lcmf/gateway/__init__.py, lcmf/analysis/transition_system.py, lcmf/cli.py, tests/test_gateway.py, tests/test_transition_system.py | 1,321 | Chapter 2 (sections 2.7-2.9: ICS protocols and existing migration approaches); Sections 4.1-4.3, 4.6 (architecture, data design, gateway); Sections 5.4, 5.7; Sections 6.4, 6.5.1-6.5.3, 6.5.8 | Slides 14-18 and 20 (gateway, migration, transition vulnerabilities, live demo) |

The comparable code sizes, the one-to-one mapping of packages to members and the shared ownership of analysis, integration and documentation reflect an equal and effective contribution by every member.

# APPENDIX C: SAMPLE OUTPUT OF THE DEMONSTRATION

The following excerpt is the unedited output of `python -m lcmf demo` (Modules 3 to 6).

**Listing C.1:** Output of the guided demonstration (excerpt)

```
================================================================================================
 Module 3 - Protocol-translation gateway (HYBRID_PINNED migration phase)
================================================================================================

  Device 1 [BRIDGED] Substation A / Feeder 1  cipher=caesar
    legacy wire  : UWX34VHT4335YOW43B39FXU4B98IUT8333SZU36649EUNFFKN339
    plaintext    : RTU01SEQ1002VLT10806CUR1865FRQ5000PWR03316BRKCCHK006
    STE (68 B)  : 4c43010101000100010000000000000000010000019bc0ab24006a7e00010000000000000001...
    STE header   : dev=1 seq=1 key_id=1 nonce=6a7e00010000000000000001
    SCADA        : ACCEPTED via secure -> V=10806 V  I=186.5 A  f=50.0 Hz  P=3316 kW
  ...

  Device 8 [NATIVE_AES] Solar Park / Feeder 8  cipher=aes-256-gcm
    STE (68 B)  : 4c43010102000100080000000000000000010000019bc0ab24005e00000800000000000001...
    STE header   : dev=8 seq=1 key_id=1 nonce=5e0000080000000000000001
    SCADA        : ACCEPTED via secure -> V=10812 V  I=268.4 A  f=50.0 Hz  P=4775 kW
================================================================================================
 Module 4 - Downlink: SCADA breaker command through the gateway
================================================================================================
  OPEN -> device 4 (BRIDGED): executed OPEN; breaker_closed=False
  OPEN -> device 7 (LEGACY_DIRECT): executed OPEN; breaker_closed=False
  OPEN -> device 8 (NATIVE_AES): executed OPEN; breaker_closed=False

================================================================================================
 Module 5 - Cryptanalysis: an eavesdropper breaks every legacy cipher
================================================================================================
Dev  Cipher     Recovered key             Result   Frames  Keys tried  Time ms
---  ---------  ------------------------  -------  ------  ----------  --------
1    caesar     3                         BROKEN   1       36          0.6
2    additive   17                        BROKEN   1       36          0.6
3    affine     (5, 8)                    BROKEN   1       432         7.6
4    vigenere   GRIDLOCK                  BROKEN   1       8           0.2
5    playfair   SUBSTATION                BROKEN   1       178         4.9
6    columnar   [3, 0, 2, 5, 4, 1, 6]     BROKEN   2       5912        87.2

================================================================================================
 Module 6 - Transition vulnerabilities (attack success, lower is better)
================================================================================================
  TV1 last-mile FDIA [gross   ] gateway only                                    forged accepted  100.0
  TV1 last-mile FDIA [stealthy] gateway only                                    forged accepted  100.0
  TV1 last-mile FDIA [gross   ] gateway + plausibility filter                   forged accepted    0.0
  TV1 last-mile FDIA [stealthy] gateway + plausibility filter                   forged accepted  100.0
  TV1 last-mile FDIA [tamper  ] native AES-GCM at the RTU (end-to-end)          forged accepted    0.0
  TV2 downgrade      HYBRID_PERMISSIVE                                        forged accepted  100.0 %
  TV2 downgrade      HYBRID_PINNED                                            forged accepted    0.0 %
  TV2 downgrade      STRICT_AES                                               forged accepted    0.0 %
  TV3 WAN replay of legacy frames        none (legacy)              replay accepted  100.0 %
  TV3 WAN replay of secure envelopes     64-bit seq + window + time replay accepted    0.0 %
  TV3 Last-mile replay through the gatew stateless gateway          replay accepted  100.0 %
  TV3 Last-mile replay through the gatew legacy sequence tracking a replay accepted    0.0 %
  TV4 tamper         Legacy Vigenere (+checksum patch)                        success           84.0 %
  TV4 tamper         AES-256-CBC, no MAC (IV bit flip)                        success          100.0 %
  TV4 tamper         AES-256-CTR, no MAC (bit flip)                           success          100.0 %
  TV4 tamper         AES-256-GCM (STE)                                        success            0.0 %
  TV5 pattern leak   AES-256-ECB                                              attacker acc.    100.0 %
  TV5 pattern leak   AES-256-GCM                                              attacker acc.     50.0 %
  TV6 nonce reuse    RAM only                                                 collisions         280
  TV6 nonce reuse    persistent (reserved blocks)                             collisions           0
  TV7 key derivation SHA-256(legacy keyword)                                  dictionary hit  GRIDLOCK
  TV7 key derivation HKDF-SHA256(random 256-bit master)                       dictionary hit    None
  TV8 overhead       legacy 52 B -> STE 68 B (+30.8 %), median gateway latency 35 us

================================================================================================
 Demonstration complete - see results/ and docs/ for the full evaluation
================================================================================================
```